Technical Reference

OpenMX 4.0 GPU Version Implementation Guide

A complete picture of the NVIDIA GPU offloading implementation and all changes from the original OpenMX 4.0

— How cuBLAS, GEMMul8, and cuSOLVER are used, and their caveats —

Target revision: OpenMX 4.0 GPU version (as of 2026-07-05, commit `8c7801fa`)

Baseline for comparison: the original OpenMX 4.0 source distribution

Verified environment: NVHPC SDK 26.3 / CUDA 13.1 / compute capability 12.0 (Blackwell-generation GeForce)

Published: July 5, 2026 (2nd edition)

> **Note:** This document describes an **unofficial** port of the first-principles code OpenMX (GPL v3) to NVIDIA GPUs; it is not an official document of the OpenMX development team. File names and line numbers refer to the revision above and may change as development proceeds.

Table of Contents

1. Introduction

1.1 Purpose and Intended Audience

This document is a public guide to the "OpenMX 4.0 GPU edition," a port of the first-principles calculation code **OpenMX 4.0**, based on density functional theory (DFT), to NVIDIA GPUs. Based on primary sources (the source code and diffs), it explains (1) the overall picture of the current implementation, (2) **every change** made relative to the original OpenMX 4.0, and (3) how to use — and what to watch out for in — the three core GPU libraries: **cuBLAS**, **GEMMul8**, and **cuSOLVER**. The intended readers are as follows.

- Users who want to run OpenMX fast on GPUs (mainly Chapters 2, 10, and 11)
- Developers who want to understand or modify this implementation (Chapters 3-9)
- Developers considering ports to other vendors such as AMD GPUs (the gpusolver layer in Chapter 5, and Chapter 11)

As prerequisites, we assume elementary knowledge of DFT and the LCAO method, the C language, MPI, and the basics of CUDA programming.

1.2 OpenMX and the Position of This Port

OpenMX (Open source package for Material eXplorer) is a DFT code using localized pseudo-atomic orbital (LCAO) basis sets and norm-conserving pseudopotentials. In addition to cluster and band calculations, it provides O(N) methods such as the divide-and-conquer (DC) method, the DC-LNO method, and the Krylov subspace method, as well as non-equilibrium Green's function (NEGF) transport calculations. It is licensed under GPL v3.

This GPU edition starts from the official OpenMX 4.0 source and adds GPU offloading incrementally over **51 commits**. The "dense-matrix eigenvalue problems" and "matrix products (GEMM)" that dominate SCF calculations, together with grid-integration and reduction computations, are moved to the GPU, while everything else (sparse-matrix processing, communication, I/O, CPU-specific paths) retains the original structure.

1.3 Overview of the Scale of Changes

Item	Value	Notes
Modified source files	37 files (+ rename of `makefile` → `Makefile` with full overhaul)	No source files were deleted
Changed lines (raw diff)	+33,974 / −20,839 lines	Includes full re-indentation (e.g. `Force.c`). Ignoring whitespace, the effective changes in the top files are on the order of 1,000-3,700 lines each
New source files	13 files, 2,555 lines total	gpusolver wrappers, the GEMMul8 bridge, GPU energy kernels, etc.
New build files	3 files	`Makefile` / `Makefile.bunshiken` / `makefile.cpu_intel.bak` (an identical copy of the original)
third_party (git submodules)	2 items	GEMMul8 (pinned to a commit), FFTW 3.3.11 (built in-tree)

1.4 Target Environment

- **Compilers:** NVIDIA HPC SDK (NVHPC) 26.3 — `mpicc` (nvc) + OpenACC, `mpif90` (nvfortran), nvcc (CUDA 13.1)

- **GPU:** CUDA-capable NVIDIA GPU. GEMMul8 uses INT8 Tensor Cores. Development and verification were done on compute capability 12.0 (Blackwell-generation GeForce)
- **Parallel model:** MPI process parallelism. GPUs are assigned round-robin by intra-node local rank, and the design assumes that **multiple ranks share one GPU**
- **OpenMP:** Builds use `-fopenmp`, but parts of the GPU path (the radial-integral offload in `Set_OLP_Kin`) require single-thread execution. When using the GPU, MPI-only operation (1 thread) is assumed

1.5 Notation

- Source locations are given as `filename:line-number` (all relative to the `source/` directory, as of 2026-07-04).
- `GPU` always executed on GPU, `conditional` opt-in via environment variables etc., `CPU` CPU execution (deliberately not GPU-ported), `dormant` implemented but not called on the current paths.
- "Original" refers to the source of the official OpenMX 4.0 distribution.

> **How to read this document:** If you only want to know how the libraries are used, see Chapters 3-5; operational parameters are collected in Chapter 10, and known pitfalls in Chapter 11. Chapter 7 gives a vertical, per-computation-path (per-solver) explanation, cross-referenced with Chapters 3-6 (a horizontal, per-library/per-technique view).

2. Overall Architecture

2.1 Design Principles

1. **Runtime switching with the CPU path fully preserved** — GPU porting is realized not by wholesale `#ifdef` replacement but by **adding branches** to the existing code. It is enabled by a single input-file line, `scf.eigen.lib gpusolver`; if not specified, the same ELPA/ScaLAPACK/LAPACK path as the original runs. The gate is concentrated in one point, the runtime flag `scf_eigen_lib_flag == GPUSOLVER` (enum value 3, `openmx_common.h:2807-2812`).
2. **Only large dense-matrix problems are GPU-ported** — when the matrix dimension is below the threshold `GPU_CPU_SWITCH_NUM = 1000` (`openmx_common.h:4079`; only DC-LNO uses its own value of 800), transfer costs dominate, so execution falls back to the CPU.
3. **Per-MPI-rank GPU use, designed for "sharing"** — round-robin assignment by intra-node local rank % number of GPUs. The situation where multiple ranks share one GPU is addressed head-on, with multiple layers of control: GPU-turn serialization, adaptive concurrency limiting, up-front decisions based on memory estimates, and retry on failure (§ 2.5).
4. **GEMMul8 is the first choice for GEMM, with automatic fallback to cuBLAS FP64** — large GEMMs are executed via FP64 emulation on INT8 Tensor Cores (Ozaki Scheme II), switching to native cuBLAS only when the workspace cannot be allocated (Chapter 4).
5. **Diagonalization uses cuSOLVER's 64-bit generic API** — centered on `cusolverDnXsyevdx` (partial eigenvalues), and the generalized eigenvalue problem $Hc = \varepsilon Sc$ is handled by a hand-rolled GPU implementation of Löwdin (symmetric) orthogonalization via the eigendecomposition of S (Chapter 5).
6. **Vendor-neutral naming (gpusolver)** — with future AMD GPU support in mind, the wrapper-layer identifiers and input keywords were renamed wholesale from `cusolver` to `gpusolver` (commits `2302f965`, `d8219384`). The cuSOLVER API calls themselves remain as implementation details of the NVIDIA build. MAGMA was also evaluated in early development but has now been completely removed from the source tree (commit `788acb4c`, "Delete unnecessary files, such as the MAGMA library"; the current source contains 0 references to MAGMA).
7. **Attention to numerical reproducibility** — the reductions in density-matrix construction keep sequential order via `#pragma acc loop seq`, spherical Bessel function tables are cached with bit-pattern-exact keys, GEMMul8 itself is bitwise reproducible, and so on — care about summation order appears throughout (on the other hand, some places are order-nondeterministic due to atomic additions; see § 11).

2.2 Layer Structure

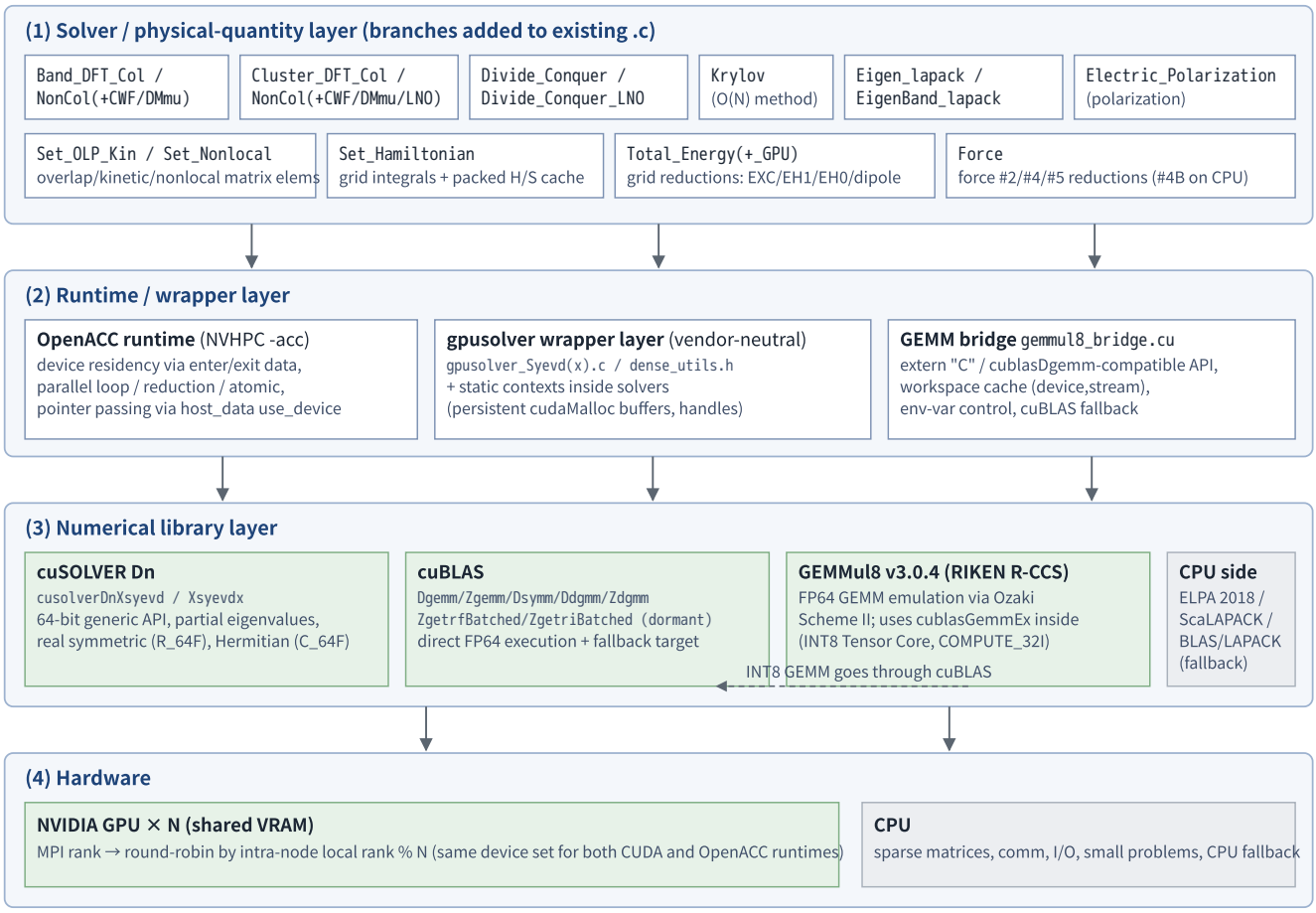

Figure 1: Layer structure of the OpenMX 4.0 GPU edition. Green = GPU execution, gray = CPU execution.

Three forms of GPU execution are used side by side.

1. **OpenACC directives** — loop parallelism for grid integrals, reductions, matrix assembly, etc. Data residency uses `enter data` / `exit data` (Chapter 6).
2. **Static contexts inside solvers + raw CUDA API** — persistent device buffers from `cudaMalloc` and cuBLAS/cuSOLVER handles are held in file-scope structs, and the libraries are called directly (`BandColGpuSolverCtx` etc.; Chapters 3 and 5).
3. **GEMMul8 via an extern "C" bridge** — a single-translation-unit scheme for calling the C++ template library from C code (Chapter 4).

2.3 Activation Flow and GPU-Offload Points within SCF

The flow until the GPU path becomes active is as follows.

1. **Input parsing** (`Input_std.c:777-791`): `scf.eigen.lib gpusolver` sets `scf_eigen_lib_flag = GPUSOLVER` . The old name `cusolver` is accepted as a deprecated alias with a warning. The user-specified value is saved in `scf_eigen_lib_flag_input` .
2. **Device initialization** (`DFT.c:52-99` ; `DFT_GPU_DeviceInit()` at the SCF entry point `DFT.c:334`): the intra-node local rank is obtained with `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` , and `cudaSetDevice` and `acc_set_device_num` are set **as a pair**. If no CUDA/OpenACC device is found, a warning is printed and the flag is automatically demoted to `scf_eigen_lib_flag = ELPA2` .

3. **Condition check at each solver entry**: the GPU dense-matrix path is entered only when `GPUSOLVER flag && matrix dimension ≥ threshold && (for some paths) matrix distribution shape conditions` are satisfied; otherwise the conventional path (ELPA2 etc.) runs unchanged.

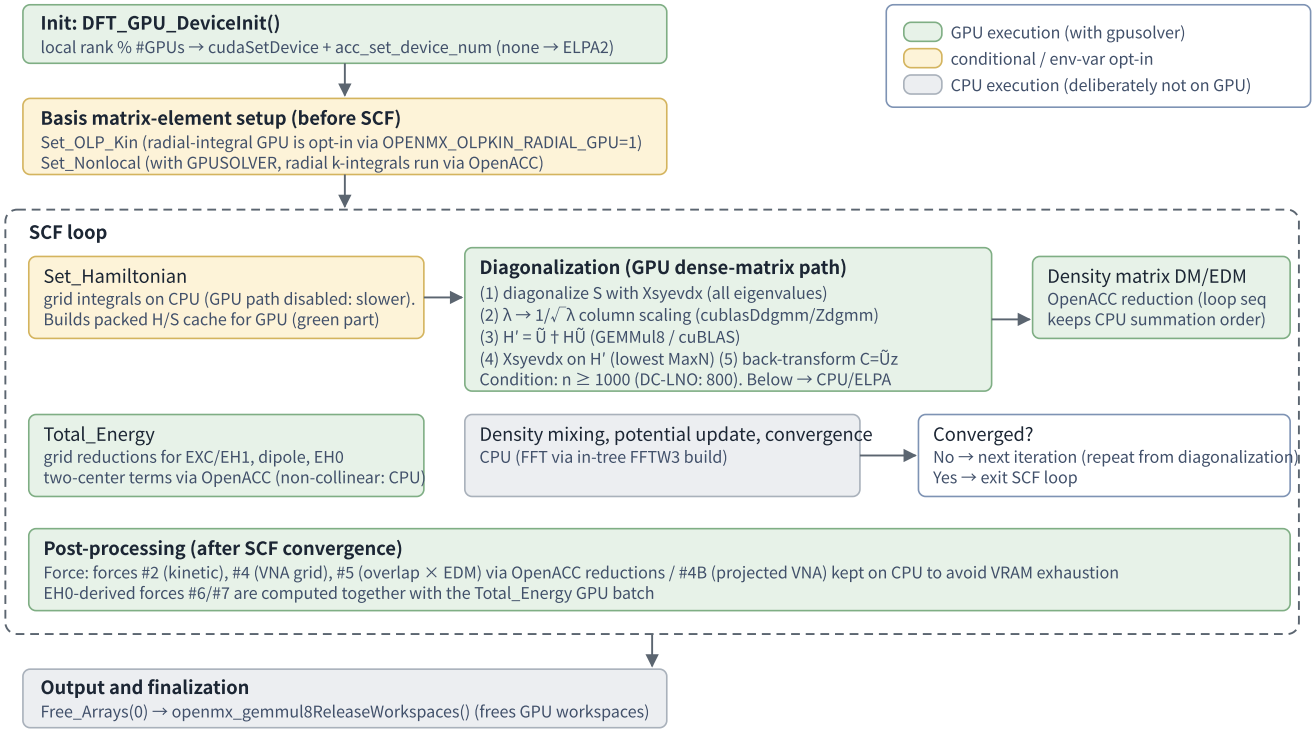

Figure 2: Flow of one SCF cycle and GPU-offload points (for collinear band/cluster calculations).

2.4 Map of GPU-Offloaded Targets

Table 2-1 shows the GPU-porting status by computation path (`scf.EigenvalueSolver`) and by auxiliary computation. Details are in Chapter 7.

Path / computation	Main file	What is GPU-ported	Activation condition
Cluster (Col)	`Cluster_DFT_Col.c`	Dense matrices are gathered onto the root rank; S diagonalization → Löwdin orthogonalization → H′ diagonalization → back-transform run on the GPU (root-dense path). GEMM uses GEMMul8 Dgemm	GPUSOLVER and n ≥ 1000
Cluster (NonCol)	`Cluster_DFT_NonCol.c`	root-dense diagonalization (Dsyevx/Zheevx) + 12 U † HU calls via GEMMul8 + DM/EDM reduction. Eigenvectors are cached and scattered from DFT.c	GPUSOLVER and 2n ≥ 1000
Band (Col)	`Band_DFT_Col.c`	The full dense-matrix diagonalization per k-point runs device-resident. GEMM uses GEMMul8 Zgemm. Caches the S transformation matrix. Supports both `all_knum==1` / `!=1` paths. GPU-turn serialization	GPUSOLVER and n ≥ 1000 (+ distribution-shape conditions)
Band (NonCol)	`Band_DFT_NonCol.c`	OpenACC-resident root-dense path. GEMM uses GEMMul8 Zgemm. Adaptive concurrency limiting based on VRAM estimates	GPUSOLVER and 2n ≥ 1000 (+ distribution-shape conditions)

Path / computation	Main file	What is GPU-ported	Activation condition
DC method (O(N))	`Divide_Conquer.c`	Per-atom sub-cluster matrices diagonalized with Xsyevdx. GEMM has a multi-stage fallback GEMMul8→cuBLAS→CPU. Up-front VRAM check	GPUSOLVER and local dimension ≥ 1000 (adjustable via `OPENMX_DC_GPU_THRESHOLD`)
DC-LNO method	`Divide_Conquer_LNO.c`	Each rank submits its own local eigenvalue problem directly to the GPU (direct per-rank dispatch). The GPU proxy mechanism is also preserved (default disabled)	GPUSOLVER and local dimension ≥ 800
Krylov method (O(N))	`Krylov.c`	Raw CUDA implementation with per-thread workspace pools. Subspace diagonalization (Xsyevd/Xsyevdx) + GEMMul8 only for large dgemm	GPUSOLVER and n ≥ 1000 (GEMM: $m \cdot n \cdot k \geq 5 \times 10^7$)
Generic eigenvalue routines	`Eigen_lapack.c` `EigenBand_lapack.c`	Branch to gpusolver_Syevdx(_Complex) when n ≥ 1000. All 20+ files calling these automatically benefit from GPU porting	GPUSOLVER and n ≥ 1000
Polarization, DMmu, CWF, LNO family	11 files	A boilerplate GPUSOLVER branch (Dsyevx/Zheevx via dense_utils) is inserted ahead of the ELPA2 branch	GPUSOLVER and n ≥ 1000 and na_rows==n etc.
Matrix-element construction	`Set_Hamiltonian.c` `Set_Nonlocal.c` `Set_OLP_Kin.c`	Radial integrals of nonlocal projectors run via OpenACC. GPU porting of Set_Hamiltonian's grid integrals was measured slower and disabled (the packed H/S cache construction supports the GPU solvers). Set_OLP_Kin is opt-in	GPUSOLVER (+ per-file conditions, § 7.7)
Total energy	`Total_Energy.c` `Total_Energy_GPU.c`	Grid reductions for EXC/EH1, dipole, CWF-Dc, and EH0 two-center terms run via OpenACC	GPUSOLVER and SpinP_switch≠3
Force	`Force.c`	Reductions for forces #2/#4/#5 run via OpenACC. #4B kept on CPU	GPUSOLVER (+ per-term conditions)
NEGF, complex matrix inversion	`Lapack_LU_inverse.c`	**dormant** A GPU LU inverse via cublasZgetrf/ZgetriBatched is implemented but the call is commented out	— (groundwork for the future)

Table 2-1: Map of GPU-offloaded targets

2.5 Measures for GPUs Shared by Multiple Ranks (Overview)

A distinctive feature of this implementation is its multi-layered defense explicitly designed for configurations like "one 16 GB-class GPU shared by 8 MPI ranks."

Mechanism	Description	Location
Allocation-failure retry	The `wait_cudafunc` macro retries CUDA/cuBLAS/cuSOLVER calls until success with random waits (≤10 µs), absorbing transient VRAM-allocation contention with other ranks	`openmx_common.h:4092-4103`
GPU-turn serialization	Band calculations introduce a running index ("turn") over "spin × k-points" and, by default every 4 turns, insert an MPI_Barrier to execute serially (a comment notes this is about 27% faster than concurrent submission with 8 ranks/GPU)	`Band_DFT_Col.c:218-355`

Mechanism	Description	Location
Adaptive concurrency limiting	Non-collinear band explicitly estimates required VRAM and, via `cudaMemGetInfo` and MPI agreement, automatically shrinks the concurrency from 4→2→1 (also applied to values explicitly set via environment variables)	`Band_DFT_NonCol.c:729-1038`
Per-turn workspace release	The ~1 GiB GEMMul8 workspace is returned at the end of each turn (default enabled)	`Band_DFT_Col.c:283-295, 2703-2705`
Up-front memory check + CPU fallback	The DC method checks free VRAM before allocating and, if insufficient, (stickily) disables the GPU path and continues on the CPU. Set_Hamiltonian also has an Allreduce check summed over sharing ranks	`Divide_Conquer.c:833-880` `Set_Hamiltonian.c:164-268`
Selecting the rank that runs the dense solver	In cluster/band, only the world representative rank becomes the owner of the dense-matrix GPU path	`DFT.c:159-204`

> **Handling of dormant code:** In this document, code whose definition exists but which is not called on the current paths (`gpusolver_Syevd` , `LU_Zinverse_GPU` , `openmx_cuda_kernels.cu` , some GEMMul8 helpers, etc.) is explicitly marked `dormant` to distinguish it. See § 11.7 for the full list.

3. How cuBLAS Is Used

3.1 Routines Used and Their Roles

cuBLAS serves three roles in this implementation: (1) direct execution of dense-matrix operations other than GEMM, such as diagonal scaling (`Ddgmm` / `Zdgmm`), (2) the FP64 fallback target when GEMMul8 cannot be used, and (3) the execution substrate for the INT8 Tensor Core GEMMs (`cublasGemmEx`) inside GEMMul8. The direct `cublasDgemm/Zgemm/Dsymm` calls that used to remain in some solvers were **all unified to go through GEMMul8** in commit `66ce1e62` (Use GEMMul8 for remaining GPU GEMM paths). Table 3-1 gives the full picture of the routines used.

cuBLAS routine	Purpose	Main call sites
`cublasDgemm` / `cublasZgemm`	FP64 fallback target inside the GEMMul8 bridge, and the second stage of the DC method's multi-stage fallback (these are the only two direct-call sites)	`gemmul8_bridge.cu:299,307,373,381`, `Divide_Conquer.c:607`
`cublasDdgmm` / `cublasZdgmm`	$1/\sqrt{\lambda}$ column scaling (product with a diagonal matrix) in the Löwdin orthogonalization	`Cluster_DFT_Col.c:261`, `Band_DFT_Col.c:1377`, `Divide_Conquer_LNO.c:520`
`cublasZgetrfBatched` / `cublasZgetriBatched`	`dormant` Complex LU factorization / matrix inverse (batch=1). Groundwork for the NEGF surface Green's function (§ 5.8)	`Lapack_LU_inverse.c:384,400`
`cublasGemmEx` (INT8, COMPUTE_32I)	Per-modulus INT8 matrix products inside GEMMul8 (never called directly from OpenMX)	`third_party/GEMMul8/src/oz2/common/matmult.hpp:16-25`

Table 3-1: List of cuBLAS routines used

3.2 Handle and CUDA Stream Lifecycles

In the main paths of the current implementation, a **file-scope static context per solver** holds one lazily initialized cuBLAS/cuSOLVER handle each (one per solver type per MPI process). They are destroyed and recreated only when the device changes.

Cluster_DFT_Col.c:118-130 — lazy initialization of the context (typical form)

```
wait_cudafunc(cudaGetDevice(&current_device));

if (ctx->initialized && ctx->device_id == current_device) return;
if (ctx->initialized) ClusterCol_GpuSolver_Destroy();

wait_cudafunc(cudaStreamCreateWithFlags(&ctx->stream, cudaStreamNonBlocking));
wait_cudafunc(cublasCreate(&ctx->cublas));
wait_cudafunc(cusolverDnCreate(&ctx->gpusolver));
wait_cudafunc(cublasSetStream(ctx->cublas, ctx->stream));
wait_cudafunc(cusolverDnSetStream(ctx->gpusolver, ctx->stream));

ctx->initialized = 1;
ctx->device_id   = current_device;
```

Note that stream handling is **asymmetric** across solvers.

- **Cluster_DFT_Col / Band_DFT_NonCol (for DM) / Krylov / DC / DC-LNO:** create a dedicated non-blocking stream, bind it via `cublasSetStream` / `cusolverDnSetStream`, and guarantee ordering of transfers with `cudaMemcpyAsync` plus `cudaStreamSynchronize` at key points.

- **Band_DFT_Col:** the context struct (`Band_DFT_Col.c:1105-1132`) has no stream member at all, and `cublasSetStream` is never called. cuBLAS runs on the default stream, and transfers use synchronous `cudaMemcpy`.
- **Ordering with OpenACC:** the OpenACC default queue and the library streams are not tied together (`acc_get_cuda_stream` / `async` clauses are unused throughout the code). Explicit synchronization is done by placing `#pragma acc wait` immediately before handing OpenACC-managed data to a library, and `cudaStreamSynchronize` immediately after (e.g. `Band_DFT_NonCol.c:433-455`).

> **Caution:** because stream assumptions differ between solvers, mixing synchronization conventions when porting a helper function from one solver to another risks race conditions. Also, each context's handles and persistent buffers are **never explicitly destroyed at normal program exit** (they are freed only on device change and at the end of a Band-family turn; at exit, cleanup is left to CUDA context teardown). The only thing `Free_Arrays(0)` frees is the GEMMul8 workspace.

3.3 Device Memory Management

- **Grow-only persistent buffers:** in the form `if (n <= ctx->matrix_dim) return;`, the buffer is reused if the required size is no larger than the existing one, and free→malloc happens only when it is insufficient (`Cluster_DFT_Col.c:146-164`, `Band_DFT_Col.c:1221-1248`). The cuSOLVER workspace works the same way (`Cluster_DFT_Col.c:192-208`).
- **Pinned memory:** almost never used. The only exception is the DC method, which uses `cudaMallocHost` on the host side (`Divide_Conquer.c:786`). Everything else is pageable malloc + (Async) Memcpy.
- **Consideration for shared GPUs:** keeping the Band-family buffers resident across SCF iterations is OFF by default (`OPENMX_BAND_GPU_PERSISTENT`). An in-code comment states explicitly that "with 16 GB shared by 8 ranks, GEMMul8 gets pushed into the FP64 fallback, which costs more than the 0.2 s per iteration saved" (`Band_DFT_Col.c:233-237`).

3.4 The Actual GEMM Engine per Solver

Since commit `66ce1e62`, **every GEMM executed on the GPU goes through the GEMMul8 bridge in all solvers** (cuBLAS FP64 is used only for the in-bridge fallback and the DC method's multi-stage fallback). However, **several helpers remain that are defined but never called**, so to avoid being misled by grep appearances, Table 3-2 lays out the paths that are actually called (the live paths).

Solver	GEMM engine on the live path	Notes (dormant code)
Band_DFT_Col	**GEMMul8** (`openmx_gemmul8Zgemm` × 3, triple-product transform and back-transform. `Band_DFT_Col.c:1452,1456,1473`)	Helper `BandCol_GpuSolver_Zgemm_OpenACC` (:1398) is unused
Band_DFT_NonCol	**GEMMul8** (via `BandNonCol_GEMMul8Zgemm_OpenACC` :433 × 6. Switched from direct cuBLAS in `66ce1e62`)	Uses the persistent workspace's cublas handle + stream. lda/ldb are computed transpose-aware
Cluster_DFT_Col	**GEMMul8** (`ClusterCol_GEMMul8Dgemm_Device` :285. H′ construction × 2 + back-transform × 1. Switched from cublasDsymm+Dgemm in `66ce1e62` — exploiting symmetry via SYMM was abandoned)	The OpenACC helper (:333) is definition-only. The old path `ClusterCol_GpuSolverDensePath` (using CPU dgemm) is unreachable

Solver	GEMM engine on the live path	Notes (dormant code)
Cluster_DFT_NonCol	**GEMMul8** (12 calls total, D/Z, U † HU transform)	—
Divide_Conquer (DC)	Multi-stage **GEMMul8 → cuBLAS → CPU** (`DC_GpuSolver_TryGpuDgemm`, `Divide_Conquer.c:575-609`)	If GEMMul8 fails, it is permanently disabled for that rank from then on
Divide_Conquer_LNO	**GEMMul8** (`openmx_gemmul8Dgemm` × 3)	—
Krylov	**GEMMul8** (`openmx_gemmul8Dgemm`, `Krylov.c:346`. GPU only when $m \cdot n \cdot k \geq 5 \times 10^7$)	Switched from cublasDgemm in `66ce1e62`
DMmu/CWF/LNO family, 11 files	GPU handles diagonalization only (GEMMs stay as before)	Each file's GEMMul8 helper is definition-only (considered preparatory)

Table 3-2: GEMM engine per solver (live paths vs. dormant code)

3.5 The Removed Generic Wrappers my_cublasDgemm / my_cublasZgemm

The self-contained GEMM wrappers `my_cublasDgemm.c` / `my_cublasZgemm.c` used in early development (`cublasCreate` → `cudaMalloc`/`cudaMemcpy` → GEMM → destroy, on every call) had long been **dormant code with zero call sites**, and were then **completely deleted** — sources, declarations, and link targets — in commit `a7f8cba6` (Remove obsolete cuBLAS wrapper files). The names appear only when reading old revisions.

A related simplification still present in the current helpers deserves attention: the in-solver GEMM helpers are **hard-fixed to alpha=1.0 / beta=0.0**, and lda/ldb handling comes in two flavors. `BandNonCol_GEMMul8Zgemm_OpenACC` / `ClusterCol_GEMMul8Dgemm_OpenACC` were fixed in `66ce1e62` to compute lda/ldb according to the transpose flags, but the live helper at `Cluster_DFT_NonCol.c:660,677` (used only with square matrices) and the dormant DMmu/CWF-family helpers still contain the **hardcoded lda=m, ldb=k**, which is correct with transposes only in the square case (m=n=k). When reusing them with non-square matrices or transposes, always verify the ld handling.

3.6 Error Handling — the wait_cudafunc Macro

Nearly every CUDA/cuBLAS/cuSOLVER call (47 sites in `Band_DFT_Col.c` alone) is wrapped in the following macro.

openmx_common.h:4079-4103 (excerpt)

```
#define GPU_CPU_SWITCH_NUM 1000
#define WAITTIME           (10.0 * 1.0E-6)

#define wait_cudafunc(call)                                          \
    while (true) {                                                   \
        if (cudaSuccess == call) {                                   \
            break;                                                   \
        }                                                            \
        double wait_time    = drand48() * WAITTIME;                  \
        double start_time   = MPI_Wtime();                           \
        double current_time = start_time;                            \
        while ((current_time - start_time) < wait_time) {            \
            current_time = MPI_Wtime();                              \
        }                                                            \
    }
```

On failure, it inserts a random 0–10 µs wait (busy-wait) and **retries the same call indefinitely until it succeeds**. Since the success code 0 is common to `cudaError_t` / `cublasStatus_t` / `cusolverStatus_t`, the three APIs can be wrapped by the same macro. This design absorbs transient `cudaMalloc` failures when multiple ranks contend for

the same GPU. An abort-style `cudacall` macro (`openmx_common.h:4082-4090`) is also defined, but it is not used on the main paths.

Responses to computational failures (e.g. eigenvalue convergence failure) are handled at the solver layer, and depending on the path they are either `MPI_Abort` / `exit` or a CPU fallback (see the table in § 5.4 for details).

3.7 Caveats When Using cuBLAS

1. `wait_cudafunc` **retries forever** — the call expression in the argument is re-evaluated on every loop. Passing it a permanent error (invalid leading dimension, lost device, etc.) results in a hang plus repeated side effects. When GPU execution "appears to be stuck", suspect this first.
2. **Fixed ld/alpha/beta in the simplified helpers** — as described in § 3.5. The live helpers' lda/ldb have been fixed to be transpose-aware, but some helpers still contain hardcoded values and will silently produce wrong results for the combination of transpose + non-square.
3. **Asymmetric stream assumptions** — Band_DFT_Col uses the default stream + synchronous memcpy; the others use dedicated streams + explicit synchronization. When porting or unifying code, check the synchronization conventions.
4. **Handles/buffers never released (a deliberate design trade-off)** — there is no explicit teardown path at process exit. If the code is ever reworked into a long-running service, release logic must be added.
5. **Zero-size matrix guard** — the GEMMul8 bridge returns success immediately for m,n,k ≤ 0 (since all GEMMs go through the bridge, this single spot protects everything).
6. **cuBLAS workspace not set** — the `cublasSetWorkspace` (32 MiB) recommended by GEMMul8 is not set on any handle (a candidate performance improvement; see the performance notes in `gemmul8.hpp`).
7. **Column-major convention** — cuBLAS is column-major, same as Fortran BLAS. For OpenMX's real symmetric matrices, some places pass the row-major flat array as-is, exploiting symmetry (it is equivalent to a transpose; `Eigen_lapack.c:1570-1574`). Hermitian matrices are explicitly repacked into column-major order (`EigenBand_lapack.c:709-714`).

4. How GEMMul8 Is Used

4.1 What GEMMul8 Is — Principle and Origin

GEMMul8 (GEMMulate) is a "GEMM emulation using INT8/FP8 matrix engines" library developed by the RIKEN Center for Computational Science (RIKEN R-CCS) (lead developer: Yuki Uchino, MIT license). This implementation incorporates **v3.0.4** (pinned to upstream commit `884a2875⋯`). Upstream repository: `https://github.com/RIKEN-RCCS/GEMMul8`.

The principle is **Ozaki Scheme II** (floating-point matrix-multiplication emulation based on integer modular techniques / the Chinese Remainder Theorem, CRT).

1. **Scaling + quantization:** the FP64 matrices A, B are exponent-adjusted and, for each of the pairwise-coprime moduli p = 256, 255, 253, 251, 247, 241, … (integers fitting in 8 bits, at most 20 of them; `src/oz2/common/table.hpp:12-31`), reduced modulo p to produce `num_moduli` INT8 matrices.
2. **Error-free integer matrix products:** an INT8 GEMM is executed per modulus. The actual engine is cuBLAS's `cublasGemmEx(⋯, CUDA_R_8I, ⋯, CUBLAS_COMPUTE_32I, ⋯)`, i.e. the **INT8 Tensor Cores (IMMA)**. Being integer arithmetic, no rounding error is introduced.
3. **CRT reconstruction:** the integer product is recovered from the per-modulus results via the Chinese Remainder Theorem, the scaling is undone, and $\alpha AB + \beta C$ is obtained.

Increasing `num_moduli` (the number of moduli) raises accuracy and lowers speed. Because the procedure is deterministic, **results are bitwise reproducible under identical settings** (stated explicitly in the upstream README). GEMMul8 is positioned as an implementation of Scheme II, an evolution of Ozaki Scheme I from the earlier line of work ozIMMU (Ootomo, Ozaki, Yokota 2024), which performs DGEMM on INT8 integer matrix engines. See the bibliography at the end for the references to cite.

4.2 Integration Architecture — the Two-File Approach

The OpenMX core is C code built with NVHPC `mpicc -std=c11` and cannot call a C++ template API directly. Moreover, the upstream build system generates a large number of objects, one per "routine × type × backend". This implementation therefore integrates via the following two files.

(a) gemmul8_openmx.cu — explicit template instantiation in a single translation unit

The upstream template **definition** headers (`gemm_impl.hpp`, `worksize_impl.hpp`, plus the 7 kernel-definition headers that upstream splits into separate objects) are included into one translation unit, and **only the 4 symbols** OpenMX needs are explicitly instantiated (a small 67-line file).

gemmul8_openmx.cu:31-44 (double version; the cuDoubleComplex version and the two workSize variants are analogous)

```
template std::vector<double> gemm<double, Backend::INT8, double, double>(
    cublasHandle_t,
    cublasOperation_t, cublasOperation_t,
    size_t, size_t, size_t,
    const double *,
    const double *const, size_t,
    const double *const, size_t,
    const double *,
    double *const, size_t,
    int, bool,
    void *const,
    void *const,
    void *const,
    bool, bool, bool, bool);
```

The four symbols instantiated are `gemm<double,INT8>`, `gemm<cuDoubleComplex,INT8>`, `workSize<false,INT8,Func::gemm>`, and `workSize<true,INT8,Func::gemm>`. This object alone is packed with `ar` to produce our own `third_party/GEMMul8/lib/libgemmul8.a` (its contents differ from the upstream library of the same name). Because this approach depends on the upstream internal header layout, **pinning the upstream commit is mandatory** (stated explicitly in a Makefile comment; § 4.7).

(b) gemmul8_bridge.cu — the extern "C" bridge

It exports 3 functions callable from the C side (prototypes in `openmx_common.h:4105-4112`).

- `cublasStatus_t openmx_gemmul8Dgemm(handle, transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)` — a signature **fully isomorphic to cublasDgemm** (alpha/beta and ld all arbitrary)
- `openmx_gemmul8Zgemm(⋯)` — the complex version (same shape)
- `openmx_gemmul8ReleaseWorkspaces()` — releases all workspaces

The bridge is responsible for (i) interpreting environment variables, (ii) querying, allocating, and caching the workspace, (iii) falling back to native cuBLAS when allocation is not possible, and (iv) calling `gemmul8::gemm`.

gemmul8_bridge.cu:296-316 (Dgemm body; Zgemm has the same structure)

```
if (gemmul8_disabled("OPENMX_GEMMUL8_DISABLE_D", "GEMMUL8_DISABLE_D")) {
    report.reason = "environment disable";
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}

cublasStatus_t status =
    ensure_workspace<false>(handle, static_cast<size_t>(m), static_cast<size_t>(n), static_cast<size_t>(k),
                            num_moduli, &work, &report);
if (status == CUBLAS_STATUS_ALLOC_FAILED) {
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}
if (status != CUBLAS_STATUS_SUCCESS) {
    return status;
}

(void)gemmul8::gemm<double, gemmul8::Backend::INT8>(handle, gemmul8_transa, gemmul8_transb, ..., num_moduli, fastmode, work);
```

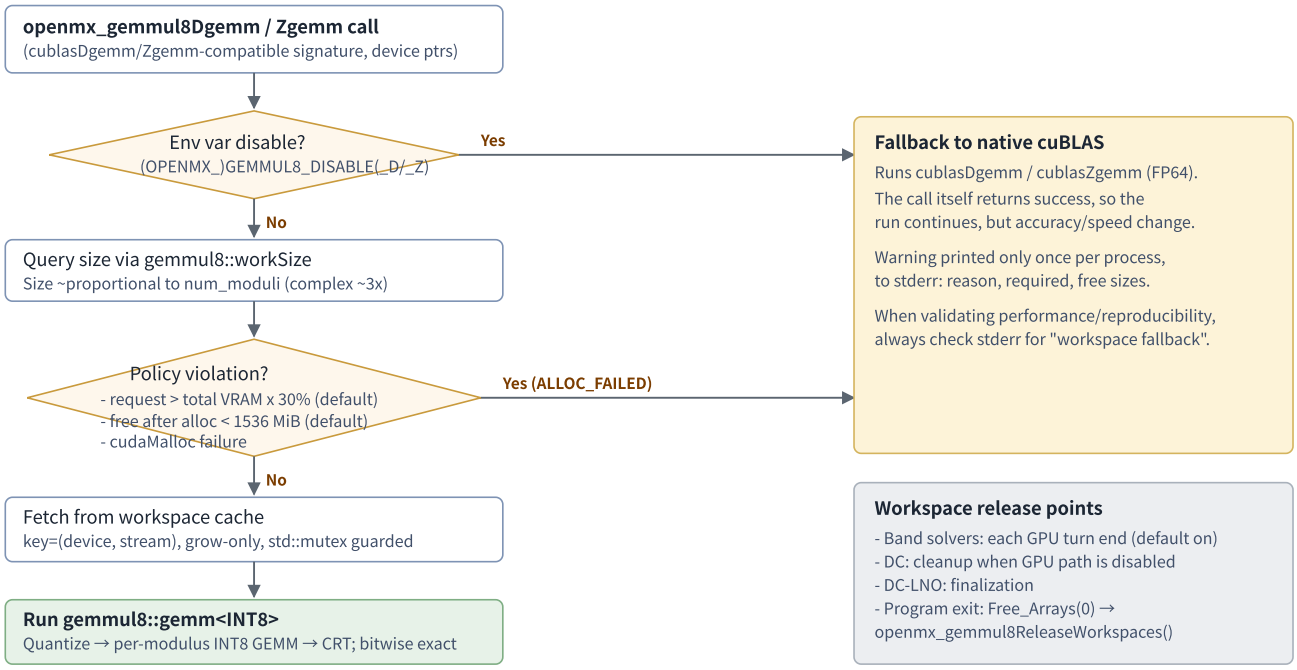

Figure 3: Dispatch flow of the GEMMul8 bridge (gemmul8_bridge.cu).

4.3 Call Parameters and Precision Settings

Parameter	Default	Environment variable (OPENMX_ name wins)	Meaning / behavior
`num_moduli`	**15**	`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	Number of moduli (precision). Valid range 2–20; out-of-range values silently revert to 15. D/Z can be set independently
`fastmode`	**false** (accurate)	`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z` ("1" to enable)	Fast scaling path. Reduces accuracy
Disable	off	`(OPENMX_)GEMMUL8_DISABLE`, `_D` / `_Z`	Pins execution to native cuBLAS FP64
WS cap ratio	**30%**	`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	Reject if the request exceeds this fraction of total VRAM (Band_DFT_NonCol setenv's 50 when unset)
Min free after alloc	**1536 MiB**	`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	Reject if free VRAM after allocation would drop below this value

- The real (D) path silently normalizes `CUBLAS_OP_C` to `CUBLAS_OP_T` (for real matrices, conjugate transpose = transpose).
- `m,n,k ≤ 0` returns success without doing anything.
- There is no code on the OpenMX side that compares or validates GEMMul8 results against a direct FP64 computation. All fallbacks are triggered by memory/environment-variable conditions, never by accuracy conditions.
- Upstream extensions such as skip-scaling are unused. The per-phase timing return value is also discarded.

4.4 Workspace Management

- **Query:** on every call, the required amount is computed via `gemmul8::workSize<is_complex,INT8>(m,n,k,num_moduli)`.
- **Cache:** a process-wide `std::unordered_map` keyed by `(device, cudaStream_t)`, protected by `std::mutex`, grow-only (`gemmul8_bridge.cu:21-44, 207-241`). Because the cuBLAS handle's stream is part of the key, note that **each handle (stream) gets its own separate region**.
- **Size scale:** A side = num_moduli INT8 matrices + shift vectors, B side likewise, C side = (num_moduli−1) intermediate matrices + a 32 MiB internal BLAS region + INT32 accumulators. Complex is roughly another factor of 3. Around n ≈ 10,000, this is **about 1 GiB per rank** (in-code comment, `Band_DFT_Col.c:285-288`).
- **Release:** only via `openmx_gemmul8ReleaseWorkspaces()`. Its call sites are (1) `Free_Arrays(0)` (at program exit, `Free_Arrays.c:24`), (2) at the end of each Band_DFT_Col GPU turn (opt out with `OPENMX_BAND_GEMMUL8_TURN_RELEASE=0`), (3) when the DC method disables its GPU path, (4) DC-LNO finalization.

4.5 Complex (Z) Support

The OpenMX side does not decompose complex matrices into real ones; it passes them as `cuDoubleComplex` directly to `gemm<cuDoubleComplex, INT8>`. GEMMul8's internal native complex path keeps 3 sets of low-precision representations per input and builds the complex product from **3 real INT8 GEMMs per modulus** (a 3M/Karatsuba-style scheme, not the 4M method; for the theoretical background see the paper on CRT-based complex matrix-multiplication emulation in the references). As a result, the complex workspace and operation count are about 3× the real case.

4.6 Which GEMMs Run through GEMMul8

There is **no** matrix-size-based dispatch inside the bridge (GEMMul8 is the default; fallbacks are triggered only by memory/environment-variable conditions). Since commit `66ce1e62`, **every GEMM on the GPU goes through the bridge** (Table 3-2). The representative example is Band_DFT_Col's generalized-eigenproblem transform H′ = Ũ † HŨ (this is the largest GEMM load).

Band_DFT_Col.c:1452-1458 — triple-product transform (first two calls shown; the single back-transform follows at :1473)

```
wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_H, n, (cuDoubleComplex *)ctx->d_S, n, &beta,
                                  (cuDoubleComplex *)ctx->d_tmp, n));

wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_S, n, (cuDoubleComplex *)ctx->d_tmp, n, &beta,
                                  (cuDoubleComplex *)ctx->d_H, n));
```

4.7 Build

- **Fetching:** `make` runs `git fetch --depth 1 origin $(GEMMUL8_COMMIT)` → `checkout` in the git submodule directory, pinning it to commit `884a2875…` (`Makefile:716-722`). The reason for the pinning is stated in a Makefile comment: "because gemmul8_openmx.cu instantiates GEMMul8 internals directly, the checkout must stay pinned to a commit whose internal layout matches".
- **Compilation:** nvcc from the CUDA 13.1 bundled with NVHPC, host compiler `-ccbin nvc++`, `-std=c++20 -O3 -diag-suppress 177 -DGPU_ARCH=$(arch) -gencode arch=compute_$(arch),code=sm_$(arch)`. The architecture is auto-

detected from `nvidia-smi --query-gpu=compute_cap` (compute_120/sm_120 in this development environment). Manual override such as `GEMMUL8_GPU_ARCH=90` is also possible (`Makefile:161-167`).

- **Environment sanitization:** `NVCC_GEMMUL8_ENV` empties `CPATH` and the like to shut out contamination from NVHPC headers and flags (`Makefile:170`).
- **Extra link flags:** `-lcublasLt -lcuda -lnvidia-ml -lstdc++` (`Makefile:172`).

4.8 Caveats When Using GEMMul8

1. **Memory usage** — the workspace is roughly proportional to num_moduli, complex is about 3×, and n ≈ 10,000 takes about 1 GiB per rank. On GPUs shared by multiple ranks, exhaustion is prevented by the three-layer scheme of "30% cap, 1536 MiB reserve, per-turn release". When changing the defaults, keep these three consistent.
2. **Silent FP64 fallback** — when the workspace is rejected, execution switches to cuBLAS FP64 with only a single warning, and **both speed and numerical results (bit patterns) change**. If things are "mysteriously slow / results wobble", check stderr for "GEMMul8 workspace fallback". For validation runs that need deterministic reproduction, either disable everything with `OPENMX_GEMMUL8_DISABLE=1` or leave ample memory headroom.
3. **Precision settings** — default is 15 moduli, accurate mode. Out-of-range settings silently revert to 15. fastmode is faster but loses scaling accuracy. D and Z can be tuned independently.
4. **First-call cost** — on the first call and whenever the size grows, GiB-scale `cudaMalloc` runs synchronously. Benchmark using the second and later iterations.
5. **Build pitfalls** — (a) unpinning the commit breaks `gemmul8_openmx.cu` via internal header layout changes. (b) Failing to clean NVHPC environment variables (CPATH etc.) can make nvcc pick up NVHPC headers and fail. (c) Forgetting to link `-lcublasLt` and `-lstdc++`. (d) Architecture auto-detection assumes `nvidia-smi` is available. (e) With a single-architecture `-gencode`, the binary is incompatible with GPUs of another generation (rebuild required).
6. **Tracking the upstream API** — the migration from the old API to the current structure (884a287) (commit `cb4d348d`) simplified `gemmul8_openmx.cu` from 149 to 67 lines. The bridge's public signatures are unchanged, with no impact on the C side. For future upstream updates too, checking "the set of definition headers the single TU includes" first is a good approach.
7. **Hardware requirements** — INT8 Tensor Cores (IMMA) are mandatory. A C++20-capable nvcc (CUDA 12 series or later) is effectively required (only CUDA 13.1 has been verified).

5. How cuSOLVER (the gpusolver Layer) Is Used

5.1 Layer Structure and Naming Intent

Eigenvalue computation uses cuSOLVER through a wrapper layer with the vendor-neutral name **gpusolver**. The layer consists of the following three tiers.

1. **Generic wrapper functions** (self-contained per call): the 5 functions in `gpusolver_Syevd.c` / `gpusolver_Syevdx.c`. There are host-array versions and `_openacc` versions that directly process OpenACC device-resident arrays (Table 5-1). As the 48-line license header at the top indicates, they originate as modifications of the syevd/syevdx samples from the NVIDIA CUDA Library Samples.
2. **Common utility header**: `openmx_gpusolver_dense_utils.h` (79 lines). It statically defines 4 functions — the 1-based/0-based variants of `OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` — and calls `MPI_Abort` if `info != 0`. It is included by 12 files in the polarization, DMmu, CWF, and LNO families.
3. **Solver-resident contexts**: `BandColGpuSolverCtx`, `ClusterColGpuSolverCtx`, `DCGpuSolverCtx`, `DCLNO_GpuSolverCtx`, the per-thread workspace pool in Krylov, and so on. They cache handles and device buffers and call the cuSOLVER API directly (the same structure as in Chapter 3).

Naming intent: the vendor neutralization is an "abstraction by naming convention"; no switching mechanism via macros or function pointers exists yet. Function names, struct member names, and input keywords are unified under gpusolver (e.g. `cusolverDnHandle_t gpusolver;`), while cuSOLVER type names and API names remain as-is as implementation details of the NVIDIA build. AMD (hipSOLVER/rocSOLVER) support is intended to be implemented separately under the same naming convention. The old input keyword `cusolver` remains as a deprecated alias with a warning. MAGMA, evaluated early in development, has been removed and is not referenced anywhere in the current source (§ 2.1).

Function (gpusolver_Syevd(x).c)	cuSOLVER API	Data location	Current callers
`gpusolver_Syevd(A,W,m)`	Xsyevd (all eigenvalues)	Built-in host⇄device transfers	dormant no callers
`gpusolver_Syevdx(A,W,m,MaxN)`	Xsyevdx (lowest MaxN)	Same as above	`Eigen_lapack.c:1576,1667`, `Cluster_DFT_Col.c:607,649`, many via dense_utils
`gpusolver_Syevdx_openacc(…)`	Xsyevdx	OpenACC resident (present + host_data)	`Eigen_lapack.c:1617,1645`
`gpusolver_Syevdx_Complex(…)`	Xsyevdx (CUDA_C_64F)	Built-in host⇄device transfers	`EigenBand_lapack.c:716`, `Band_DFT_Col.c:1419`, many via dense_utils
`gpusolver_Syevdx_Complex_openacc(…)`	Xsyevdx (CUDA_C_64F)	OpenACC resident	`EigenBand_lapack.c:677,741`

Table 5-1: Public API of the generic wrappers

5.2 Details of the APIs Used

- Everything uses the **generic (X) API, 64-bit integer version**: `cusolverDnXsyevd(_bufferSize)` / `cusolverDnXsyevdx(_bufferSize)`. The legacy API (`cusolverDnDsyevd` etc.) and the `Zheevd`-family aliases are never

used; complex cases also use the unified form of passing `CUDA_C_64F` to Xsyevdx. The generalized eigenvalue solver `cusolverDnXsygvd` is not used either (§ 5.3).

- Common parameters: `params = NULL`, `jobz = CUSOLVER_EIG_MODE_VECTOR`, `uplo = CUBLAS_FILL_MODE_LOWER`, range selection via `CUSOLVER_EIG_RANGE_I` (il=1, iu=MaxN). Some implementations switch to `RANGE_ALL` when `m == MaxN`, while others always use RANGE_I — both coexist.
- Handles: the generic wrappers do `cusolverDnCreate` → create a non-blocking stream → compute → release everything, on every call. Resident contexts cache them (§ 3.2).
- Workspace: every path queries both the device and host requirements with `_bufferSize` before allocating. Resident contexts use a grow-only cache. Only the DC method performs a preflight check via `cudaMemGetInfo` before allocation, and if memory is insufficient it disables the GPU path and continues on the CPU (`Divide_Conquer.c:833-880`).

gpusolver_Syevdx.c:103-117 — the core call form (64-bit generic API)

```
wait_cudafunc(cusolverDnXsyevdx_bufferSize(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu,
                                           1L, MaxN, &h_meig, CUDA_R_64F, d_W, CUDA_R_64F,
                                           &workspaceInBytesOnDevice, &workspaceInBytesOnHost));

wait_cudafunc(cudaMalloc((void **)(&d_work), workspaceInBytesOnDevice));
h_work = (workspaceInBytesOnHost == 0) ? NULL : malloc(workspaceInBytesOnHost);
...
wait_cudafunc(cusolverDnXsyevdx(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu, 1L, MaxN,
                                &h_meig, CUDA_R_64F, d_W, CUDA_R_64F, d_work, workspaceInBytesOnDevice, h_work,
                                workspaceInBytesOnHost, d_info));
```

5.3 Handling the Generalized Eigenvalue Problem Hc = εSc

The cuSOLVER generalized solver is not used; instead, each path implements OpenMX's traditional **Löwdin (symmetric) orthogonalization via the eigendecomposition of S** by itself on the GPU. Cholesky decomposition is never used. The procedure is the following 6 steps, common to all paths (see also Figure 4).

1. Diagonalize S with Xsyevdx (all eigenvalues) → eigenvectors U, eigenvalues λ
2. Treat small/negative eigenvalues (**differs by path**: Band/Cluster round up to 1×10^{-10}, DC discards values below `OLP_eigen_cut`, DC-LNO applies `fabs`)
3. Scale the columns of U by $1/\sqrt{\lambda}$ (`cublasDdgmm/Zdgmm`) → $\tilde{U}$
4. Build $H' = \tilde{U}^{\dagger} H \tilde{U}$ with 2 GEMMs (GEMMul8; Table 3-2)
5. Diagonalize H' with Xsyevdx (il=1..MaxN) → eigenvalues ε, eigenvectors z
6. Back-transform $c = \tilde{U} z$ with a GEMM

Cluster_DFT_Col.c:249-262 — S diagonalization and $1/\sqrt{\lambda}$ scaling (Löwdin orthogonalization on the GPU)

```
if (rebuild || !(ctx->transformed_s_valid && ctx->transformed_s_dim==n)){
    ClusterCol_GpuSolver_EigenDevice(ctx->d_S,n,n,ko0+1);

    for (int l=1; l<=n; l++){
        if (ko0[l]<1.0e-10) ko0[l] = 1.0e-10;
        ko0[l] = 1.0/sqrt(ko0[l]);
    }

    wait_cudafunc(cudaMemcpyAsync(ctx->d_W,ko0+1,sizeof(double)*(size_t)n,cudaMemcpyHostToDevice,ctx->stream));

    old_s = ctx->d_S;
    new_s = ctx->d_tmp;
    wait_cudafunc(cublasDdgmm(ctx->cublas,CUBLAS_SIDE_RIGHT,n,n,
                              old_s,n,ctx->d_W,1,new_s,n));
```

> **Caution (differences in numerical behavior):** because the treatment of small eigenvalues of S (round-up / discard / fabs) is not unified across paths, for systems with ill-conditioned overlap matrices the numerical behavior of the same physical system may differ subtly depending on the solver path.

How the number of partial eigenvalues MaxN is chosen: Band (Col) uses `MaxN = (TZ-charge)/2 + max(0.4·Ne/2, 100)` (`Band_DFT_Col.c:1813-1818`); Cluster solves for all eigenvalues in the first SCF iteration, and afterwards `(TZ-charge)/2 + max(0.1n-0.2n, 400)` (`Cluster_DFT_Col.c:357-379`). Solving only for the occupied bands plus a margin keeps the cost of syevdx down.

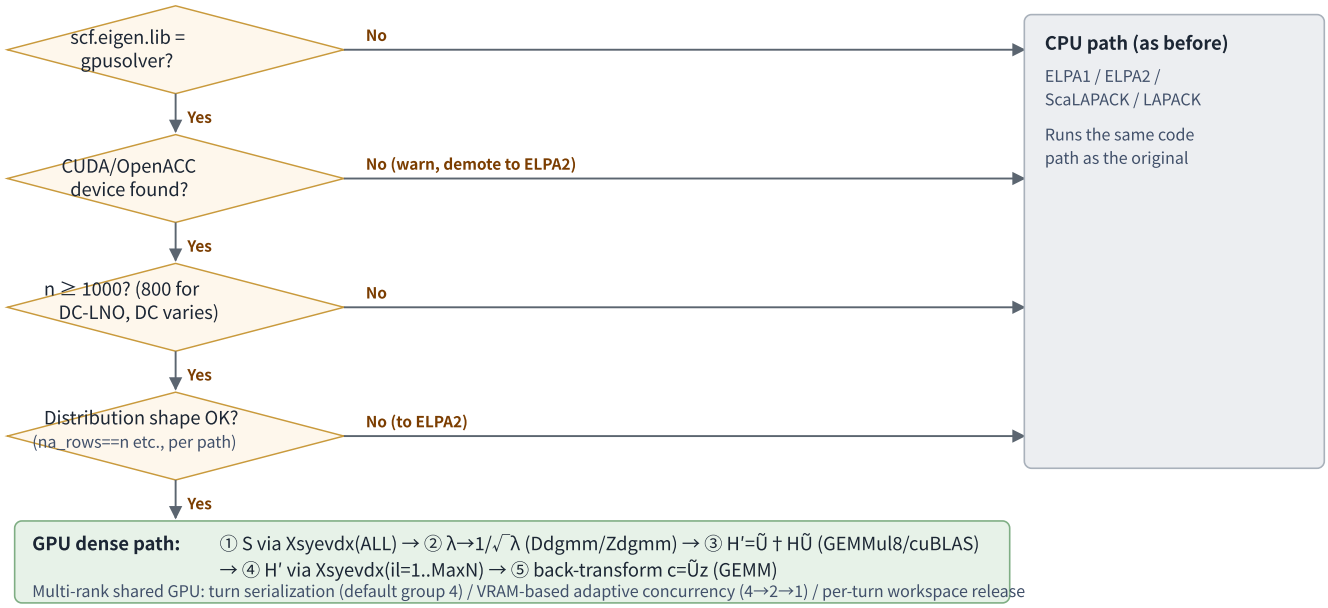

Figure 4: Eigenvalue solver dispatch and the GPU dense-matrix path.

5.4 devInfo Checking and Failure Handling (Per Path)

`d_info` is allocated on the device in every path and copied back to the host for inspection after the computation, but **the failure policy differs per path**. Assume the following table when making modifications.

Path	info check	Behavior on failure
Generic wrappers (gpusolver_Syevdx etc.)	Only returned to caller	Caller-dependent
Via dense_utils header (11 files)	Yes	Immediate `MPI_Abort`

Path	info check	Behavior on failure
`Eigen_lapack.c:1590-1593`	**info<0 only** exit(10)	info>0 (convergence failure) passes through. However, the higher-level `Eigen_lapack()` has a retry with an eigenvector orthogonality check, cycling the solver over to the CPU family and recomputing on failure (up to 3 times)
`EigenBand_lapack.c:25-32`	Yes	Fall back to the CPU Householder method `Eigen_HH`
Band_DFT_Col / Cluster_DFT_Col	Yes (Band also checks h_meig)	Immediate abort (exit / MPI_Abort)
Divide_Conquer (DC)	Yes (also h_meig)	Sets a **sticky disable flag**; from then on that rank uses the CPU DC solver. GEMMul8 workspace is also released
Divide_Conquer_LNO	Yes (h_meig not checked)	exit(10) (no fallback)
Krylov	Yes (also h_meig)	Falls back on the spot to CPU LAPACK (`Eigen_lapack2`)

Table 5-2: Per-path differences in devInfo checking and failure handling

5.5 Concurrency Control — the GPU-Turn Mechanism and Adaptive Limits

5.5.1 Band (Col): Serialization of GPU Turns

When multiple ranks (the representative ranks of spin worlds × k-point worlds) submit dense solves to the same GPU simultaneously, VRAM is exhausted, GEMMul8 drops to its FP64 fallback, and everything slows down. Therefore a global sequence number over "spin × k-point" — the **GPU turn** — was introduced, serialized by default (per an in-code comment: with 8 owner ranks per GPU this is about 27% faster than concurrent submission).

Band_DFT_Col.c:2637-2647 — skeleton of turn serialization

```
my_gpu_turn = spin * T_knum + kloop;
const int serialize_gpu_turns = BandCol_SerializeGpuSolverGpuTurns();
const int gpu_turn_group = serialize_gpu_turns ? BandCol_GpuTurnGroup() : 1;
const int first_gpu_turn = serialize_gpu_turns ? 0 : my_gpu_turn;
const int last_gpu_turn = serialize_gpu_turns ? Num_Comm_World1 * T_knum : (my_gpu_turn + 1);
for (int gpu_turn = first_gpu_turn; gpu_turn < last_gpu_turn; gpu_turn++) {
    if (serialize_gpu_turns && gpu_turn % gpu_turn_group == 0) {
        MPI_Barrier(mpi_comm_level1);
    }

    if (owns_global_dense_rank && my_gpu_turn == gpu_turn) {
```

The number of turns allowed to run simultaneously (the group width) is determined in the priority order `OPENMX_BAND_GPU_TURN_GROUP` → `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE` → default 4. The first diagonalization loop for `all_knum != 1` is controlled by a separate knob, `OPENMX_BAND_GPU_MAX_CONCURRENT_K` (default 4). The latter was **clamped to a maximum of 4** by commit `8c7801fa` (Limit band GPU k-turn concurrency); specifying anything higher is rounded down to 4.

5.5.2 Band (NonCol): Adaptive Limits Based on VRAM Estimation

With commit `43e30c65` (Limit band GPU solver concurrency adaptively), the non-collinear band path replaced the fixed value with concurrency determined by "an explicit estimate of the required device memory × the measured free capacity".

1. **Requirement model** (`Band_DFT_NonCol.c:767-817`): estimates the peak of the 3 phases — overlap / Hamiltonian / wavefunction. The syevdx workspace is included by actually querying `_bufferSize` (falling back to $4 \cdot n^2 \cdot 16$ bytes if the query fails).
2. **Check**: tests whether "per-turn requirement × number of concurrent ranks + reserve (**a full extra turn-sized cushion** for OpenACC/GEMMul8 scratch)" fits in the free VRAM.
3. **Decision**: if it does not fit, scale down in steps 4→2→1, then take `MPI_Allreduce(MIN)` over the communicator of ranks sharing the same GPU (`MPI_Comm_split_type(SHARED)` + device ID), so all ranks agree on **a unanimous limit value**.

Furthermore, with commit `8c7801fa`, this adaptive limit **is also applied to the explicitly specified values (capped at 4)** of `OPENMX_BAND_GPU_MAX_CONCURRENT_K` / `MAX_RANKS_PER_DEVICE` (always routed through `BandNonCol_AutoGpuTurnLimit`). Even if a large value is given via environment variable, it is automatically reduced if it does not fit in VRAM.

Band_DFT_NonCol.c:847-859 — the check (with original comment)

```
required = required_bytes*(size_t)concurrent_ranks;
reserve = BandNonCol_RootDenseReserveBytes(total_bytes,required);

/* Dense non-collinear turns also use OpenACC and GEMMul8 scratch space
   around the explicitly estimated matrices. Keep a full extra turn-sized
   cushion so the automatic limiter does not choose a group that only
   fits on paper and then fails in cuMemAlloc. */
if (reserve<required) reserve = required;

if (group_required!=NULL) *group_required = required;
if (reserve_bytes!=NULL) *reserve_bytes = reserve;

return (required<=free_bytes && reserve<=free_bytes-required);
```

5.5.3 DC-LNO: direct per-rank dispatch

With commit `f508a2a6` (Enable direct GPUSOLVER dispatch for DC-LNO), DC-LNO switched from a **proxy scheme** in which "the GPU owner rank handles other ranks' eigenvalue problems on their behalf via MPI" to a scheme in which "**every rank submits its own local problem (dimension ≥ 800) directly to the GPU**". The proxy mechanism is preserved for experiments behind `DCLNO_ENABLE_SHARED_GPU_PROXY 0` (`Divide_Conquer_LNO.c:82`). The number of GPUs used within a node can be capped with the input keyword `scf.Gpu.Num`.

Divide_Conquer_LNO.c:509-529 (excerpt) — S diagonalization → scaling → GEMMul8 transform → H diagonalization all on the GPU

```
DCLNO_GpuSolver_Eigen(ctx->d_S, NUM, NUM, ko + 1);

for (l = 1; l <= NUM; l++) {
    ko[l] = 1.0 / sqrt(fabs(ko[l]));
}
wait_cudafunc(cudaMemcpyAsync(ctx->d_W, ko + 1, ..., cudaMemcpyHostToDevice, ctx->stream));

wait_cudafunc(cublasDdgmm(ctx->cublas, CUBLAS_SIDE_RIGHT, NUM, NUM,
                          ctx->d_S, NUM, ctx->d_W, 1, ctx->d_tmp, NUM));
wait_cudafunc(openmx_gemmul8Dgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, NUM, NUM, NUM, &alpha, ctx->d_H, NUM,
                                  ctx->d_tmp, NUM, &beta, ctx->d_S, NUM));
wait_cudafunc(openmx_gemmul8Dgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, NUM, NUM, NUM, &alpha, ctx->d_tmp, NUM,
                                  ctx->d_S, NUM, &beta, ctx->d_H, NUM));

DCLNO_GpuSolver_Eigen(ctx->d_H, NUM, NUM2, ko + 1);
```

5.6 MPI Rank → GPU Device Assignment

- **Collective version** (`set_cuda_default_device_from_local_rank.c:34-56`): obtains the local rank within a node via `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` and calls `cudaSetDevice` with `local_rank % deviceCount`. All ranks must call it together (calling from only one side deadlocks).
- **Noncollective version** (:58-73): for the hot path within SCF. Obtains the local rank from environment variables in the order `OMPI_COMM_WORLD_LOCAL_RANK` → `MV2_COMM_WORLD_LOCAL_RANK` → `SLURM_LOCALID` → `PMI_LOCAL_RANK`. If none are set, the global rank is used as a substitute (in that case the assignment can be skewed on multi-node runs).
- **The OpenACC side is set in tandem** (`set_openacc_device_from_local_rank.c`): calls `acc_set_device_num` under the same rule. The design **requires setting the same device in both the CUDA and OpenACC runtimes**.

5.7 LU_Zinverse_GPU — a Dormant Implementation for NEGF

> **Dormant code:** the GPU version of complex LU inversion, `LU_Zinverse_GPU` (`Lapack_LU_inverse.c:347-444`), is implemented not with cuSOLVER but with the **cuBLAS batched API** `cublasZgetrfBatched` / `cublasZgetriBatched` (batch=1). A is an OpenACC `deviceptr`, singular-matrix detection aborts, and the result is written back with `acc kernels`. However, the call from the public function `Lapack_LU_Zinverse` is commented out as `/* LU_Zinverse_GPU(n, A, handle); */` (:63), so the default remains the traditional CPU implementation (LAPACK zgetrf/zgetri). The callers are the NEGF surface Green's function computation (`TRAN_Calc_SurfGreen.c` and others), so this is **groundwork for a future GPU port of the NEGF-family complex matrix inversion**.

5.8 Caveats When Using cuSOLVER (the gpusolver Layer)

1. **Estimating workspace size** — one n×n complex matrix takes $16n^2$ bytes, and the solver itself plus workspace plus GEMM scratch requires several times that. The design assumption in the code comments is "a 16 GB-class GPU shared by 8 ranks". If the system is large relative to VRAM, adjust the concurrency knobs (§ 5.5, Chapter 10) first.
2. **Infinite-loop risk of `wait_cudafunc`** — same as § 3.7. Hangs on a permanent error.
3. **Non-uniform devInfo checking** — as in Table 5-2, the recoverability from convergence failures differs by path. The Band/Cluster/DC-LNO/dense_utils families are "failure = immediate termination".
4. **Presence or absence of CPU fallback** — only DC (sticky disable), Krylov, EigenBand_lapack (→Eigen_HH), and Eigen_lapack (retry cycling) have runtime fallbacks. Only when no device is present does the whole run demote to ELPA2.
5. **Symmetry and 0/1-based conversion** — every call assumes symmetric/Hermitian with uplo=LOWER. The 0/1-based shift of the eigenvalue array (`W[l]=W[l-1]`) is needed in many places, absorbed by dense_utils.
6. **Inconsistent handling of the condition number of S across paths** — see the warning in § 5.3.
7. **Reproducibility** — the transform GEMMs run through GEMMul8, so the rounding behavior differs from cuBLAS FP64, and memory-condition-dependent fallbacks can change the path from run to run (mitigations in § 4.8-2).
8. **The Band GPU path for all_knum≠1** requires `na_rows==n && na_cols==n` and aborts if not satisfied. The polarization and DMmu families silently fall back to ELPA2 when the same condition is not met.

6. OpenACC versus Raw CUDA

6.1 Overall Picture

Loop-parallel offloading is written in NVHPC's OpenACC (`-acc -Minfo=accel`). Key points:

- **There are no GPU-only `#ifdef` guards**. Because CUDA headers (`cublas_v2.h` etc.) are included unconditionally in `openmx_common.h:5-7`, **this tree cannot be built CPU-only** (for CPU use the original or `makefile.cpu_intel.bak`). All gating is done via runtime flags.
- There is no `-gpu=ccXX` specification; the target CC relies on auto-detection of the GPU on the build machine. **Managed memory is not used**. The `async` clause and `acc_get_cuda_stream` are also unused throughout the code.
- About 26 files contain `#pragma acc`. The heaviest are `Band_DFT_NonCol.c` (63 occurrences), `Divide_Conquer.c` (56), `Cluster_DFT_NonCol.c` (48), `Band_DFT_Col.c` (46), `Force.c` (40). **`Krylov.c` has zero OpenACC** (compiled without `-acc`, raw CUDA only).
- The basic form is `parallel loop gang` + `loop vector reduction`. Reductions where reproducibility matters, such as density matrix construction, use `loop seq` (sequential) so the summation order matches the CPU.

6.2 Three Data-Residency Patterns

1. **Per-call temporary data regions (copyin/copyout)** — `Set_*`, `Force`, `Total_Energy_GPU`. Because OpenMX's multi-dimensional pointer arrays cannot be traversed on the device, the design is unified so that **data is always repacked into 1-D flat arrays on the host before transfer**.
2. **SCF-scope residency via enter data / exit data + update** — mainly the non-collinear solvers. Example: `Cluster_DFT_NonCol.c` does `enter data create` on S → diagonalize and scale on the device → `update self` → `exit data delete`. Eigenvectors stay resident until DM construction.
3. **Raw `cudaMalloc` + `deviceptr` bridge** — `Band_DFT_Col.c` / `Cluster_DFT_Col.c` run the sparse→dense scatter-add against the static context's `d_H/d_S` (cudaMalloc regions) from the OpenACC side with `#pragma acc parallel loop deviceptr(d_H)` + `acc atomic update`. This is the only pattern where OpenACC-managed memory and cudaMalloc memory coexist within a single kernel.

Passing pointers to libraries is done with `host_data use_device` (23 sites in total). After declaring residency with `present`, inside the block the array name is replaced by its device address and can be passed straight to cuSOLVER/cuBLAS/GEMMul8 (no H2D/D2H transfers).

gpusolver_Syevdx.c:143-151 — typical form of passing an OpenACC-resident array directly to cuSOLVER

```
#pragma acc data      present(A[0 : m * m])
#pragma acc data      present(W[0 : MaxN])
#pragma acc host_data use_device(A, W)
    {
        cusolverDnHandle_t cusolverH = NULL;
        wait_cudafunc(cusolverDnCreate(&cusolverH));
        cudaStream_t stream = NULL;
        wait_cudafunc(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
        wait_cudafunc(cusolverDnSetStream(cusolverH, stream));
```

> **Caution (present errors):** the `_openacc`-style wrappers assume the caller has already established the data region. Calling without it crashes with an NVHPC runtime FATAL (present table lookup failed). The current code is unified under the convention that "the caller always issues enter data first".

6.3 Synchronization Design

The OpenACC default queue and the CUDA streams used by libraries are not connected. Ordering is guaranteed solely by explicit synchronization: (1) `#pragma acc wait` just before a library call, (2) `cudaStreamSynchronize` on the library stream, (3) synchronous `cudaMemcpy`. This design prioritizes correctness and simplicity over performance.

6.4 openmx_cuda_kernels.cu — a Verified But Not-Wired-In Collection of Raw CUDA Kernels

> **Important (accurate current status):** `openmx_cuda_kernels.cu` / `.h` (696+103 lines) is a module that rewrites the OpenACC compute regions of `Cluster_DFT_NonCol.c` / `Set_Hamiltonian.c` / `Set_OLP_Kin.c` as raw CUDA kernels. However, as of 2026-07-04 it is **untracked in git, not a Makefile build target, and has 0 call sites from the .c side**; **the current binary runs the corresponding OpenACC versions**. Bitwise-match verification is complete, but it is not wired in (a future replacement candidate).

The design policy is stated in the comment at the top of the file (verbatim): "every reduction that was sequential in the OpenACC code is kept sequential per thread here, so the floating point summation order is unchanged; 1.0/sqrt(x) is used instead of the approximate rsqrt intrinsic" — that is, bitwise reproduction relative to the CPU through **order-preserving sequential reduction** and **avoidance of the approximate rsqrt instruction**. Main kernels:

Kernel / wrapper	Computation	Reduction design
`dense_scatter_add_kernel`	Sparse→dense scatter-add (zero-fill + add)	atomicAdd (nondeterministic order)
`scale_columns_invsqrt_kernel`	Column scaling for S^−1/2 construction	No reduction. `1.0/sqrt(w)` (no rsqrt)
`build_ss2_kernel` / `assemble_hs2_kernel`	Assembly of non-collinear 2n×2n Ss2/Hs2	No reduction
`calc_dm_noncol_kernel<CALC_EDM>`	10 DM/EDM components from dense eigenvectors	**State-k loop sequential in 1 thread** (same order as the old OpenACC loop seq)
`setham_base_kernel` / `setham_me_kernel`	Base coupling of H / grid-integration matrix elements	Grid-point loop sequential in 1 thread
`olpkin_radial_kernel` + table-residency helpers	Radial k integration (overlap and kinetic matrix elements)	k-grid loop sequential in 1 thread. Radial tables device-resident for process lifetime

As common infrastructure it has a grow-on-demand `DeviceBuffer` cache, fixed block=256, and `cudaGetLastError`+`cudaDeviceSynchronize` at the end of every wrapper (freely mixable with cuSOLVER/cuBLAS on other streams, as stated explicitly in the header). Note that the current OpenACC version, e.g. `Set_OLP_Kin.c:601`, uses `loop vector reduction` (parallel reduction), which follows a different summation-order convention than the sequential reduction of this CUDA version.

6.5 NVHPC-Specific Caveats — Real Examples of Miscompilation Avoidance

The OpenACC of NVHPC 26.3 (`-acc`) has known defects that break specific code, and the Makefile works around them by **removing `-acc` on a per-file basis** (with reason comments). These are points that must be re-verified whenever the NVHPC version is changed.

File	Compile setting	Reason from the Makefile comment (summary)
`Occupation_Number_LDA_U.c` / `Eff_Hub_Pot.c` / `Total_Energy.c`	`CC_NOACC` (-O1, no -acc)	NVHPC 26.3 + -acc miscompiles the occupation-number/potential paths of LDA+U Type2/cFLL
`zero_cfrac.c`	`CC_NOACC`	-acc makes DSYGV fail and breaks the poles of the ON2 method
`RestartFileDFT.c`	`CC_NOACC`	Can segfault in restart I/O of over-decomposed MPI runs
`MD_pac.c`	`CC_NOACC`	Can segfault in the post-SCF MD control path
`Krylov.c`	`CC_NOACC_O3` (-O3, no -acc)	GPU acceleration is done via the raw CUDA API only (no OpenACC)
`Band_DFT_NonCol.c` / `Divide_Conquer.c`	`CC3` (-O2)	Optimization-level adjustment
`truncation.c` / `OutData.c`	`CC2` (-O1)	Same as above
`DFTD3vdW_init.c`	`CC_DFTD3` (-O0)	Same as above

Table 6-1: Per-file compilation adjustments (including NVHPC 26.3 defect workarounds)

In addition, the need to build `Total_Energy.c` without `-acc` gave rise to a structure in which only the OpenACC kernel part is split into a separate translation unit, `Total_Energy_GPU.c` (built with `-acc`) (§ 7.8). Also, `Force.c` contains a spot that re-enables OpenMP parallelism conditionally with `#if NVHPC_VERSION >= 250000` as a workaround for an old NVHPC OpenMP bug. The `if (use_dc_openacc)` guard at `Divide_Conquer.c:3695` still carries the comment "// compiler's bug".

7. GPU Implementation Details by Computation Path

7.1 Band Calculation (Collinear) — Band_DFT_Col.c

The largest modification target (effectively about 3,700 changed lines). The condition for the GPU dense-matrix path is `scf_eigen_lib_flag==GPUSOLVER && n ≥ 1000` (plus distribution-shape conditions). Highlights:

- **Dense-matrix construction cache:** The part that assembles the per-k-point dense matrices from the sparse matrices (H, S) is ported to OpenACC, and a cache keyed by a "fingerprint" of the pair structure (`entries` / `phase_r` / `phase_i`) is kept resident with `enter data`. Only the phases are refreshed via `update device`. The scatter-add uses `deviceptr` + `acc atomic update` (`Band_DFT_Col.c:1539-1622`).
- **Uses the packed H/S cache from Set_Hamiltonian:** The matrix aggregation before diagonalization is performed directly from the packed format built by Set_Hamiltonian (§ 7.7).
- **Diagonalization and transforms:** The S transformation matrix (Ũ) is computed once at the first pass, saved to the host-side `transformed_s`, and reused across SCF iterations (`Band_DFT_Col.c:1138-1177, 2649-2684`). H′ is built with GEMMul8 Zgemm × 3 (§ 4.6), and diagonalization uses `cusolverDnXsyevdx`. The eigenvectors stay resident on the device until the DM phase (comment :2708).
- **`all_knum != 1` path:** By design, S is re-diagonalized on every turn to reduce memory (an intentional memory/compute trade-off inherited from the original). This path requires `na_rows==n && na_cols==n`.
- **GPU turn serialization and VRAM measures:** As described in §5.5.1 and §2.5. Keeping device buffers resident across SCF iterations is OFF by default; even when turned ON, it is automatically disabled if free memory falls below 2048 MiB. The GEMMul8 workspace is released after each turn.
- **DM phase:** Uses an OpenACC `gang` + `loop seq` reduction so the summation order matches the CPU (`Band_DFT_Col.c:967-1001`).

7.2 Band Calculation (Non-collinear) — Band_DFT_NonCol.c

- An OpenACC resident-style (`enter data`-centric) root-dense path. Dense-matrix construction with phases (scatter-add), S diagonalization + S^−1/2, assembly of the 2n×2n spinor H2/S2, U † HU (GEMMul8 Zgemm × 2), eigenvector back-transform, and DM/EDM (`loop seq`) all run on the GPU.
- Since `66ce1e62`, GEMM uses GEMMul8 (Table 3-2). Ordering with respect to the OpenACC queue is explicitly controlled via `#pragma acc wait` → `openmx_gemmul8Zgemm` → `cudaStreamSynchronize` (`Band_DFT_NonCol.c:433-455`).
- Has **adaptive concurrency limiting (§5.5.2)** and a check for whether multiple k-worlds may run concurrently (Allreduce SUM of the required totals vs. Allreduce MIN of the minimum free memory; if it does not fit, it prints "Serializing non-collinear dense GpuSolver k-worlds" and serializes).
- Has a dedicated function that queries only `Xsyevdx_bufferSize` with a temporary handle for GPU memory estimation (:729-765).

7.3 Cluster Calculations — Cluster_DFT_Col.c / Cluster_DFT_NonCol.c

- **Col:** `ClusterCol_GpuSolverRootDensePath` (:448-545) is the active path. Only rank 0 of a spin world aggregates the dense matrices and solves on the GPU (S diagonalization → Ddgmm → GEMMul8 Dgemm × 2 → Xsyevdx → back-transform also via GEMMul8 Dgemm; switched from cublasDsymm+Dgemm in `66ce1e62`), then distributes the eigenvectors via MPI_Send/Recv. The S transformation matrix is cached on the device. The

older implementation `ClusterCol_GpuSolverDensePath` (GPU only for S diagonalization, GEMM via CPU dgemm) remains in the code but is unreachable.

- **NonCol:** root-dense diagonalization (real Dsyevx + complex Zheevx). U † HU uses GEMMul8 D/Zgemm, 12 calls in total (`Cluster_DFT_NonCol.c:1355-1393`). The dense eigenvectors `dense_evec` are cached resident via `enter data create` , and later stages receive them via `Cluster_DFT_NonCol_ScatterGpuSolverCachedEVec` called from `DFT.c` . DM/EDM uses `loop seq` reductions.
- MaxN (number of partial eigenvalues) is the full count only in the first SCF iteration; afterwards it is the occupied count plus a margin (§ 5.3).

7.4 The DC Method — Divide_Conquer.c

- Each rank independently processes the per-atom truncated cluster matrices (local dimension NUM). The GPU condition is `NUM ≥ DC_GPU_Threshold()` (default 1000, changeable via `OPENMX_DC_GPU_THRESHOLD`).
- **The Col side is a raw CUDA implementation:** `DCGpuSolverCtx` holds cudaMalloc buffers plus pinned host buffers (`cudaMallocHost` , the only usage in this implementation). S diagonalization (up to the 2nd SCF iteration) → H′ construction (`DC_GpuSolver_TryGpuDgemm` : GEMMul8 → cublasDgemm on failure → CPU if that also fails) → the active subspace is sliced within the device via `cudaMemcpy2DAsync` → Xsyevdx.
- **Multi-layered safety mechanisms:** a VRAM preflight before allocation (`GPUSOLVER_MIN_FREE_AFTER_MB` , default 1536 MiB), detection of host malloc failure, and detection of eigendecomposition failure — any of these raises a **sticky disable flag**, after which that rank continues with the CPU DC solver (the GEMMul8 workspace is also released). This is the most defensive implementation among the GPU-enabled paths.
- **The NonCol side is an OpenACC implementation:** the subspace transforms (S transpose, H·U·M1, M1·U † HU·M1, back-transform) run with `kernels loop independent collapse` , and diagonalization goes through `Eigen_gpusolver_x_complex_openacc` . The old OpenACC regions on the Col side are hard-disabled with `use_dc_openacc = 0` (comment: "compiler's bug").

7.5 The DC-LNO Method — Divide_Conquer_LNO.c

- The current scheme is **direct per-rank dispatch** (§ 5.5.3): every rank executes `cudaSetDevice` and submits its own local problem (NUM ≥ 800 = `GPU_CPU_SWITCH_NUM2`) directly to the GPU. Workspace sized for Max_Msize is pre-allocated per rank.
- The old **GPU proxy scheme** (rank 0 of a group of ranks sharing the same GPU performs diagonalizations on behalf of the others via MPI tags 41001-41004) is retained under `DCLNO_ENABLE_SHARED_GPU_PROXY 0` .
- The number of GPUs per node is capped by `scf.Gpu.Num` (default 30 ≒ effectively unlimited). A communicator `DCLNO_gpu_group_comm` is built for groups of ranks sharing the same GPU.
- There is no fallback on failure — exit(10). h_meig is unchecked (Table 5-2).

7.6 The Krylov Method — Krylov.c

- **No OpenACC; raw CUDA only** (compiled with `CC_NOACC_O3` , i.e. without `-acc`). Holds a per-OpenMP-thread `Krylov_GPU_Workspace` pool (d_A/d_B/d_C/d_W/d_work + a dedicated non-blocking stream).
- `Krylov_Dgemm` (:311): H2D Async → `openmx_gemmul8Dgemm` → D2H → StreamSynchronize (switched from cublasDgemm in `66ce1e62`). If the FLOP count $m \cdot n \cdot k < 5 \times 10^7$, CPU BLAS is used (`KRYLOV_GPU_DGEMM_MIN_FLOPS`).

- `Krylov_Eigen2` (:354): solves the standard eigenvalue problem of the embedded Hamiltonian with Xsyevd if n==EVmax, or Xsyevdx for a partial spectrum. n < 1000 goes to the CPU. On failure it falls back on the spot to `Eigen_lapack2` (CPU).
- Since the S transformation is already handled during Krylov basis construction (CPU `Inverse_S_by_Cholesky`), the GPU side deals only with the standard eigenvalue problem.

7.7 Matrix-Element Construction — Set_Hamiltonian.c / Set_Nonlocal.c / Set_OLP_Kin.c

Set_Hamiltonian.c (effectively about 1,800 changed lines)

- **The OpenACC version of the grid integration is implemented but permanently disabled** (`Set_Hamiltonian_MatrixElements_OpenACC_Enabled()` does `return 0`). The reason is stated in a comment: packing and transfer costs dominate, making it slower than the CPU. The GPU version of the base combination (H = F_Kin·H0 + F_VNA·HVNA + F_NL·HNL + …) is also opt-in via `OPENMX_SETHAM_BASE_GPU=1` because of a measured "10 ms vs 2 ms on the CPU".
- **The main contribution is the packed H/S cache:** a new API family that holds H and S in a 1-D packed format for GPUSOLVER (`Set_Hamiltonian_Build_GpuSolver_HS_Cache` and others, declared in `openmx_common.h:3339-3351`). The Band/Cluster solvers use this cache directly for the dense-matrix assembly before diagonalization, accelerating transfer and assembly on the GPU path. It is prepared each SCF iteration from `DFT.c:207-214`.
- **Memory safety mechanism:** `Set_Hamiltonian_DeviceMemoryOK` (:164-268) estimates the required amount and, within the group of ranks sharing the same GPU, performs an Allreduce (SUM of requirements vs. MIN of free memory); if insufficient, it falls back to the CPU path with a warning. OpenACC execution is restricted to world representative ranks (`DFT.c:159-204`).

Set_Hamiltonian.c:1409-1424 (excerpt) — sequential reduction of the grid integration (implemented but currently disabled OpenACC version)

```
#pragma acc loop vector
for (e = 0; e < (size_t)spin_count * mat_size; e++) {
    ...
    double sum = hbuf[hidx];
    int Nog;
#pragma acc loop seq
    for (Nog = 0; Nog < NOLG; Nog++) {
        sum += vpotbuf[...] * orbs0buf[...] * orbs1buf[...];
    }
    hbuf[hidx] = sum;
}
```

Set_Nonlocal.c (+1,356/−306)

- The radial k integration of the nonlocal projector matrix elements <ϕ|P> is ported to OpenACC (`collapse(2)` + `loop vector reduction`). Always enabled under GPUSOLVER. Bessel function tables are recomputed only when the species changes.
- In addition, hardening was carried out that replaces fixed-size stack arrays with heap allocations with overflow checks (`SetNonlocal_MallocArray` etc.).

Set_OLP_Kin.c (+529/−181) and Spherical_Bessel.c

- Offloading the radial integration of the overlap and kinetic-energy matrix elements is measured to be slower than the CPU due to being data-transfer-bound (comment: about 1.2 s vs 0.3 s), so it is **opt-in** via `OPENMX_OLPKIN_RADIAL_GPU=1`. It additionally requires a thread count of 1 (`Nthrds==1`).
- As an effective CPU-side speedup, a **spherical Bessel function table cache keyed by exact bit-pattern match of the pair distance r** was added (results are bitwise identical; single-thread only). `Spherical_Bessel.c` also had its per-call malloc/free removed from the hot path.

7.8 Total Energy — Total_Energy.c + Total_Energy_GPU.c

Reason for the file split: Because `-acc` in NVHPC 26.3 miscompiles the LDA+U path, the `Total_Energy.c` body has to be built without `-acc` (`CC_NOACC`). The OpenACC kernels alone were therefore split into a separate translation unit, `Total_Energy_GPU.c` (with `-acc`, 517 lines). The gate is `TotalEnergyUseOpenACC()`: disabled for non-collinear (SpinP_switch==3), otherwise follows the GPUSOLVER flag.

Four kinds of grid reductions are GPU-enabled (all using a temporary data region with copyin on every call):

1. **EXC/EH1 grid integration** (Ena, Eef, EH1, and EXC are reduced in a single `parallel loop reduction(+:5 variables)`)
2. **The 6 dipole-moment components** (grid coordinates GN→(n1,n2,n3) are reconstructed on the device)
3. **CWF double-counting correction** (dcEH1/dcEXC)
4. **Batched computation of the EH0 two-center terms** — all atom pairs are put into arrays at once, and the per-species spherical grids, VPS meshes, and V_H atom tables are flattened and transferred. A `gang` (pairs) × `vector reduction` (grid points) layout computes the energy and the r derivative (equivalent to forces #6/#7) simultaneously. Spline interpolation is implemented as a device function with `#pragma acc routine seq`.

Total_Energy_GPU.c:39-47 — typical form of a grid reduction

```
#pragma acc data copyin(Density_Grid_B0[0:grid_count], Density_Grid_B1[0:grid_count], \
                        ADensity_Grid_B[0:grid_count], PCCDensity_Grid_B0[0:grid_count], \
                        PCCDensity_Grid_B1[0:grid_count], dVHart_Grid_B[0:grid_count], \
                        Vxc_Grid_B0[0:grid_count], Vxc_Grid_B1[0:grid_count], \
                        RefVxc_Grid_B[0:grid_count], VNA_Grid_B[0:vna_grid_count], \
                        VEF_Grid_B[0:vef_grid_count])
  {
#pragma acc parallel loop reduction(+:local_Ena,local_Eef,local_EH1,local_EXC0,local_EXC1)
    for (BN=0; BN<grid_count; BN++){
```

7.9 Forces — Force.c

- The reductions for force #2 (kinetic-energy term Tr[DM·dH0]), #4 (VNA grid term), and #5 (overlap term −Tr[EDM·dS]) are ported to OpenACC. The common helper copyins "per-atom offset tables + flattened (weight, hx, hy, hz)" and obtains the forces with a two-stage `gang` (atoms) × `vector reduction` (terms) reduction.
- **Force #4B (projected VNA) is intentionally kept on the CPU.** Original comment: "In DC-LNO/GPUSOLVER runs many MPI ranks can share one GPU while GPUSOLVER still owns most device memory, so these transient copies can exhaust device memory. Keep Force4B on the CPU trace path" (`Force.c:5588-5596`).
- This file has the largest number of changed lines in the diff (over ±11,000 raw lines), but most of that is a full re-indentation; the effective change is about 3,700 lines.

Force.c:166-183 (excerpt) — two-stage gang (atoms) × vector (terms) reduction

```
#pragma acc parallel loop gang
for (Mc_AN = 0; Mc_AN <= num_atoms; Mc_AN++) {
    double sumx = 0.0;  double sumy = 0.0;  double sumz = 0.0;
    if (Mc_AN != 0) {
        size_t start = atom_offsets[Mc_AN - 1];
        size_t end = atom_offsets[Mc_AN];
        size_t idx;
#pragma acc loop vector reduction(+:sumx, sumy, sumz)
        for (idx = start; idx < end; idx++) {
            double weight = weights[idx];
            sumx += weight * vx[idx];
            sumy += weight * vy[idx];
            sumz += weight * vz[idx];
        }
    }
    Fx[Mc_AN] = sumx;  Fy[Mc_AN] = sumy;  Fz[Mc_AN] = sumz;
}
```

7.10 Derived Solvers — Polarization, DMmu, CWF, LNO (11 files with a boilerplate pattern)

The following 11 files apply the same boilerplate pattern: include of `openmx_gpusolver_dense_utils.h`, device assignment when GPUSOLVER is requested, a static context, and insertion of a `scf_eigen_lib_flag==GPUSOLVER && GPU_CPU_SWITCH_NUM<=n && na_rows==n && na_cols==n` branch **immediately before the ELPA2 branch** (diagonalization via `OpenMX_GpuSolver_DenseDsyevx/DenseZheevx`). If the conditions are not met, control flows to the conventional ELPA/ScaLAPACK.

File	Changed lines	GPU diagonalization insertion points
`Band_DFT_Col_CWF.c`	+74/−6	Zheevx × 3 (S, H, CWF band)
`Band_DFT_Col_DMmu.c`	+75/−5	Zheevx × 4
`Band_DFT_NonCol_CWF.c`	+76/−11	Zheevx × 3
`Band_DFT_NonCol_DMmu.c`	+76/−5	Zheevx × 4
`Cluster_DFT_Col_CWF.c`	+64/−5	Dsyevx × 2
`Cluster_DFT_Col_DMmu.c`	+63/−4	Dsyevx × 2
`Cluster_DFT_Col_LNO.c`	+66/−5	Dsyevx × 2
`Cluster_DFT_NonCol_CWF.c`	+65/−2	Dsyevx × 1 + Zheevx × 1
`Cluster_DFT_NonCol_DMmu.c`	+66/−3	Dsyevx × 1 + Zheevx × 1
`Electric_Polarization_BandCol.c`	+27/−5	Dsyevx × 1 + Zheevx × 4 (DM, overlap, H, ABC)
`Electric_Polarization_BandNonCol.c`	+40/−14	Zheevx × 5 (also includes adjustments to the fallback conditions on the ELPA1 side)

Table 7-1: Files with the boilerplate GPUSOLVER branch inserted

7.11 Generic Eigenvalue Routines — Eigen_lapack.c / EigenBand_lapack.c

- `Eigen_lapack()` (real symmetric) branches to `Eigen_gpusolver_x` (→ `gpusolver_Syevdx`) when `flag != ELPA1/2 && n ≥ 1000`. **The 20+ files that call Eigen_lapack (orbital optimization, DC, LNO, etc.) automatically benefit from the GPU port.** A caller-level solver rotation retry with an eigenvector orthogonality check sits above it.

- `EigenBand_lapack()` (Hermitian) goes to `gpusolver_zheevx` when N $\geq$ 1000. On failure it falls back to the CPU Householder method `Eigen_HH`.
- Both have `_openacc` entry points that operate directly on OpenACC device-resident arrays, used from paths such as the non-collinear DC OpenACC path. The repacking between 2-D arrays and 1-D device arrays is also done on the device.

8. Changes Unrelated to GPU Offloading

8.1 OpenMP Bug Fixes — Eliminating Heap Corruption from Missing private Declarations

In the original OpenMX 4.0, `Stress.c` and `Total_Energy.c` had `#pragma omp parallel` constructs whose `private()` clauses did not include the loop variable `i`, resulting in **a race where multiple threads simultaneously read and write the implicitly shared `i`**. If one thread advances `i` right after another thread has evaluated the loop bound, an out-of-bounds array access occurs, manifesting as heap corruption (corruption of malloc metadata). This was identified as the cause of runtest crashes during development of this GPU version and fixed (commit `d451eeb8`).

Stress.c:729 — the fix (appending `,i` at the end of the private list; Total_Energy.c:2085 has the same form)

```
/* before the fix (original) */
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(...),sumt1,sumt2,sumt3,sumt4,sumt5)

/* after the fix (GPU version) */
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(...),sumt1,sumt2,sumt3,sumt4,sumt5,i)
```

- `Stress.c`: The race site is `for (i=0; i<9; i++){ sumt1[i]=0.0; ⋯ }` inside the parallel region (executed on every pass of the atom loop). This one line is the only diff in this file.
- `Total_Energy.c`: The Lebedev spherical-grid integration parallel section in `Calc_EXC_EH1()`. The racing loop is `for (i=0; i<(FNAN[Gc_AN]+1); i++)`, which indexes into `Dr[i][ir][ia]` and the like, so a race can point outside the allocated region (the heap corruption path).

> **Significance:** These two bugs also exist in the original OpenMX 4.0 and can manifest with OpenMP thread parallelism (thread count ≥ 2) independently of the GPU. The fixes are also worth applying when using the CPU version of OpenMX.

8.2 Memory Management and Hardening

File	Fix
`openmx.c`	Calls `Raise_Stack_Limit()` immediately at startup to automatically raise the stack soft limit to the hard limit (`getrlimit/setrlimit(RLIMIT_STACK)`). A countermeasure against crashes from large stack arrays in environments where `ulimit -s` is not set
`truncation.c` / `Free_Arrays.c`	Adds NULL assignment after freeing `VNA_Grid` / `VNA_Grid_B` / `VEF_Grid` / `VEF_Grid_B` (double free prevention)
`Set_Nonlocal.c`	Replaces fixed-size arrays with heap allocations with overflow checks (§ 7.7)
`Free_Arrays.c`	Calls `openmx_gemmul8ReleaseWorkspaces()` at exit (wherefrom==0) (GPU-related)

openmx.c:75-88 — Raise_Stack_Limit (full text)

```
static void Raise_Stack_Limit(void)
{
#ifdef RLIMIT_STACK
  struct rlimit limit;

  if (getrlimit(RLIMIT_STACK,&limit)!=0) return;
  if (limit.rlim_cur==RLIM_INFINITY) return;
  if (limit.rlim_max==0 || limit.rlim_cur==limit.rlim_max) return;

  limit.rlim_cur = limit.rlim_max;
  setrlimit(RLIMIT_STACK,&limit);
#endif
}
```

8.3 Other Fixes

File	Fix
`Runtest.c`	(1) Tightened input-file detection from partial matching via `strstr` to suffix matching `has_dat_suffix()` (prevents false positives such as `foo.dat~`). (2) Deletes the old `<System.Name>.out` before running each test to prevent contamination from previous results (with MPI_Barrier)
`Spherical_Bessel.c`	Removed per-call malloc/free (recurrence coefficient arrays) from the hot path and inlined the coefficient computation (formulas unchanged)
`openmx_common.h`	Added an include guard (absent in the original). Changed the version string from "4.0" to "4.0 GPU"
`Input_std.c`	Added GPU-related keywords (Chapter 10)
`mpao.c`	Only a one-line diff of `#include <ctype.h>` (a utility outside the build; no impact)

9. Build System

9.1 Complete Toolchain Overhaul

The original `makefile` (Intel oneAPI + MKL) was completely rewritten as `Makefile` (NVHPC + CUDA + self-built FFTW + GEMMul8). The original is preserved byte-identically as `makefile.cpu_intel.bak`.

Item	Original (makefile)	GPU version (Makefile)
C compiler	`mpiicx -O3 -xHOST -qopenmp`	`mpicc` (nvc) from the OpenMPI bundled with NVHPC + `-acc -Minfo=accel -std=c11 -mtune=native -march=native -fopenmp`
Fortran	`mpiifx`	`mpif90` (nvfortran) `-O2 -fopenmp`
BLAS/LAPACK	MKL (static)	Static `liblapack_lp64.a` / `libblas_lp64.a` bundled with NVHPC
ScaLAPACK	MKL ScaLAPACK	`libscalapack_lp64.a` from the HPC-X OpenMPI side
FFT	MKL FFTW wrapper	Self-built static FFTW 3.3.11 (third_party)
GPU libraries	—	`-lcudart -lcusolver -lcublas -lcublasLt -lcuda -lnvidia-ml -lnvJitLink -cuda` + `libgemmul8.a`
SDK assumptions	Intel oneAPI	`NVHPC_ROOT ?= /opt/nvidia/hpc_sdk/Linux_x86_64/26.3`, `NVHPC_CUDA_VERSION ?= 13.1` (overridable via variables)

Build environment isolation: `Makefile:69-104` pins `PATH` to NVHPC only and `unexport`s more than 30 environment variables such as `CPATH` / `LD_LIBRARY_PATH` / `CC`. CFLAGS and the like from the user environment have no effect (intentionally).

Makefile:131-140 — compiler definitions (excerpt)

```
COMMON_INCLUDES = $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include -I$(NVHPC_CUDA_TARGET)/include -I$(NVHPC_CUDA_HOME)/include
COMMON_CFLAGS   = -w -Minfo=accel -acc -std=c11 -mtune=native -march=native -fopenmp $(COMMON_INCLUDES)

CC  = $(MPICC) $(COMMON_CFLAGS) -O3
CC2 = $(MPICC) $(COMMON_CFLAGS) -O1
CC3 = $(MPICC) $(COMMON_CFLAGS) -O2
CC_NOACC = $(filter-out -Minfo=accel -acc,$(CC2))
CC_NOACC_O3 = $(filter-out -Minfo=accel -acc,$(CC))
CC_DFTD3 = $(MPICC) -w -O0 -std=c11 -fopenmp $(COMMON_INCLUDES)
FC = $(MPIF90) -O2 -mtune=native -march=native -fopenmp $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include
```

Whereas the original used a single `$(CC)` for all files, the GPU version applies **per-file optimization and OpenACC adjustments** (Table 6-1). There is no OpenACC target architecture specification (`-gpu=ccXX`); the GPU of the build machine is auto-detected. To cross-target, add it to `COMMON_CFLAGS`.

9.2 Link Configuration

Makefile:142-150, 172 (excerpt)

```
NVPL_SCALAPACK_LIB ?= $(NVHPC_MPI_LIBDIR)/libscalapack_lp64.a
NVPL_LAPACK_LIB    ?= $(NVHPC_COMPILER_LIB)/liblapack_lp64.a
NVPL_BLAS_LIB      ?= $(NVHPC_COMPILER_LIB)/libblas_lp64.a
NVPL_LIBS          ?= -Wl,--start-group $(NVPL_SCALAPACK_LIB) $(NVPL_LAPACK_LIB) $(NVPL_BLAS_LIB) -Wl,--end-group
CUDA_LIBS          ?= ... -Wl,--no-as-needed -lnvJitLink -Wl,--as-needed -lcudart -lcusolver -lcublas -cuda
LIB = $(FFTW3_LIBS) $(NVPL_LIBS) $(MPI_FORTRAN_LIBS) -pgf90libs -mp -lpthread -lm -ldl $(CUDA_LIBS)
...
LIB += -lcublasLt -lcuda -lnvidia-ml -lstdc++
```

The link line is `$(CC) $(OBJS) $(GEMMUL8_LIB) $(LIB) -lm -o openmx` (`Makefile:337-338`). Additions to OBJS: `Total_Energy_GPU.o`, `gemmul8_bridge.o`, `gpusolver_Syevd.o gpusolver_Syevdx.o set_cuda_default_device_from_local_rank.o set_openacc_device_from_local_rank.o`, and `OutData_Binary.o` (a dead link with no callers). The formerly included `my_cublasDgemm.o my_cublasZgemm.o` were removed in `a7f8cba6`. **`openmx_cuda_kernels.o` is not included** (§ 6.4).

9.3 Automatic Builds of third_party

Because the default target is `openmx: $(FFTW3_LIB) $(GEMMUL8_LIB) $(OBJS)`, a bare `make` builds everything fully automatically in the order FFTW3 → GEMMul8 → the main program. Both are git submodules (`.gitmodules`: RIKEN-RCCS/GEMMul8, FFTW/fftw3).

FFTW 3.3.11

- Since the submodule's git source lacks the generated codelets, the build checks for a sentinel file and, if absent, supplements only the missing files via `rsync --ignore-existing` from `https://fftw.org/fftw-3.3.11.tar.gz` (also committed to the repository).
- It verifies that the checkout is exactly the tag `fftw-3.3.11` and runs `autoreconf` if necessary. An out-of-tree build (`build/fftw3`) runs `configure --enable-static --disable-shared` (with nvc/nvc++/nvfortran as compilers) → `make install` into `third_party/fftw3-install`.

GEMMul8

- Pinned fetch/checkout to commit `884a2875⋯` (rationale in § 4.7). The upstream build system is not used; the single file `gemmul8_openmx.cu` is compiled with nvcc and `ar rcs` produces `libgemmul8.a` in-house.

Makefile:716-727 — fetching GEMMul8 and the single-TU build

```
$(GEMMUL8_DIR)/Makefile:
	mkdir -p $(GEMMUL8_REPO_DIR)
	cd $(GEMMUL8_REPO_DIR) && \
	  { [ -d .git ] || git init -q .; } && \
	  { git remote get-url origin >/dev/null 2>&1 || git remote add origin $(GEMMUL8_REPO_URL); } && \
	  git fetch --depth 1 origin $(GEMMUL8_COMMIT) && \
	  git checkout -q $(GEMMUL8_COMMIT)
$(GEMMUL8_OPENMX_OBJ): gemmul8_openmx.cu $(GEMMUL8_DIR)/Makefile
	$(NVCC_GEMMUL8_ENV) $(NVCC) $(NVCC_GEMMUL8_FLAGS) -c gemmul8_openmx.cu -o $(GEMMUL8_OPENMX_OBJ)
$(GEMMUL8_LIB): $(GEMMUL8_OPENMX_OBJ)
	mkdir -p $(GEMMUL8_DIR)/lib
	ar rcs $(GEMMUL8_LIB) $(GEMMUL8_OPENMX_OBJ)
```

`clean` was extended to include `fftw3-clean` (removes build/fftw3 and fftw3-install) and `gemmul8-clean` (removes libgemmul8.a). In other words, after `make clean` the third_party components must be rebuilt.

9.4 ELPA (elpa-2018.05.001)

The `elpa-2018.05.001/` directory is **completely identical to the original** (no diff). The object list is also identical, and the kernels remain the generic CPU versions (**ELPA's GPU kernels are not used**). The only difference is that the Fortran compiler changed from mpiifx to nvfortran. Even in the GPU version, ELPA is still actively used for systems with n < 1000 and on paths that do not meet the GPU conditions, so preserving it is important.

9.5 Derived Makefiles

- **`makefile.cpu_intel.bak`** — byte-identical copy of the original makefile (preserves the CPU/Intel configuration).
- **`Makefile.bunshiken`** — variant for a shared compute cluster (module environment: `nvhpc/25.9-nompi` + `openmpi/5.0.8` + `gcc-toolset/15`). It is **one generation older than the main Makefile** in the following respects: NVHPC 25.9 + external OpenMPI, CUDA 13.0, g++ as the nvcc host compiler, GEMMul8 in the old layout (not pinned to a commit), and object names still using the pre-gpusolver-rename `cusolver_*.o`. It serves as a reference example when porting to other environments.

9.6 Build Procedure and Caveats

1. `git clone --recursive` (submodules: fftw3, GEMMul8. Even without GEMMul8, make fetches it automatically).
2. Run `make` in `source/` (default goal: openmx). This executes everything in one pass: FFTW3 supplementation → configure → install, GEMMul8 compilation → ar, and compilation and linking of the main program.
3. `make install` copies the binary to `../work/openmx`. Utilities are built with `make all`.

> **Build caveats:** (1) Changing the NVHPC version requires re-verifying the miscompilation workarounds in § 6.5. (2) The HPC-X paths hardcode a version string (hpcx-2.25.1) and must be updated when the SDK is upgraded. (3) The GEMMul8 architecture can be overridden, e.g. `make GEMMUL8_GPU_ARCH=90` (the default is nvidia-smi auto-detection). (4) The auxiliary targets present in the original, such as the API library (libopenmx_api) and elpa_col, have been removed. (5) Parallel make (-j) is possible as far as the dependency graph is concerned, but in this project's workflow it is used only when explicitly specified.

10. Runtime Control Reference

10.1 Input File Keywords

Only the following two input keywords were added or extended for GPU functionality (all other tuning is done via environment variables).

Input_std.c:777-791 — added portion (in full)

```
/* "cusolver" is a deprecated alias of "gpusolver"; ... */
s_vec[0]="elpa2"; s_vec[1]="lapack"; s_vec[2]="elpa1";    s_vec[3]="gpusolver"; s_vec[4]="cusolver";
i_vec[0]=ELPA2;   i_vec[1]=0;        i_vec[2]=ELPA1;       i_vec[3]=GPUSOLVER;   i_vec[4]=-GPUSOLVER;
input_string2int("scf.eigen.lib", &scf_eigen_lib_flag, 5, s_vec,i_vec);
if (scf_eigen_lib_flag==-GPUSOLVER){
  if (myid==Host_ID){
    printf("Warning: scf.eigen.lib=cusolver is deprecated; use scf.eigen.lib=gpusolver instead.\n");
  }
  scf_eigen_lib_flag = GPUSOLVER;
}
scf_eigen_lib_flag_input = scf_eigen_lib_flag;

/* use GPU? (added by H.Kawai, February 2024) */
input_int("scf.Gpu.Num", &SCF_Gpu_Num, 30);
```

Keyword	Type	Default	Meaning
`scf.eigen.lib`	string choice	`elpa2`	Adds **`gpusolver`** (enables the entire GPU path) to the existing `elpa2` / `lapack` / `elpa1`. `cusolver` is a deprecated alias that emits a warning. Automatically demoted to ELPA2 when no GPU is present
`scf.Gpu.Num`	int	30	Upper limit on the number of GPUs used within a node. Currently used only to trim the GPU count for DC-LNO (the default of 30 effectively means "use all detected GPUs")

10.2 Environment Variables

GPU execution tuning is by design done via environment variables (adjustable from the job script without changing the input file). For variables that exist in both an `OPENMX_`-prefixed and a `GEMMUL8_`-prefixed form, the OPENMX_ version takes precedence.

Environment variable	Default	Meaning (where defined)
GEMMul8 (gemmul8_bridge.cu)		
`(OPENMX_)GEMMUL8_DISABLE` / `_D` / `_Z`	off	Disable GEMMul8 and pin to cuBLAS FP64 (globally / real only / complex only)
`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	15	Number of moduli (precision). Range 2–20; out-of-range values revert to 15
`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z`	0	Fast scaling mode (reduced accuracy)
`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	30	Upper limit on workspace as a fraction of total VRAM (Band_DFT_NonCol setenvs 50 if unset)
`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	1536	Minimum free VRAM that must remain after workspace allocation
Concurrency and residency control for band calculations (Band_DFT_Col.c / Band_DFT_NonCol.c)		
`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`	1	Enable/disable GPU turn serialization (0 for concurrent submission)
`OPENMX_BAND_GPU_TURN_GROUP`	4	Number of GPU turns allowed to run simultaneously (group width)

Environment variable	Default	Meaning (where defined)
`OPENMX_BAND_GPU_MAX_CONCURRENT_K`	4	Number of simultaneous k-points in the first diagonalization loop for `all_knum≠1`. Clamped to a maximum of 4 in `8c7801fa`. In NonCol, the adaptive limit (§ 5.5.2) also applies on top of the specified value
`OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE`	—	Common fallback value for the two knobs above (simultaneous ranks per GPU)
`OPENMX_BAND_GPU_PERSISTENT`	0	Device buffer residency across SCF iterations (not recommended on shared GPUs)
`OPENMX_BAND_GPU_PERSISTENT_MIN_FREE_MB`	2048	Minimum free VRAM required to allow residency
`OPENMX_BAND_GEMMUL8_TURN_RELEASE`	1	Per-turn release of the GEMMul8 workspace (0 to retain)
`OPENMX_BAND_PROFILE`	0	Display performance measurement counters
DC method (Divide_Conquer.c)		
`OPENMX_DC_GPU_THRESHOLD`	1000	Minimum local cluster dimension for GPU offloading
`(OPENMX_)GPUSOLVER_MIN_FREE_AFTER_MB`	1536	Minimum free VRAM after eigenvalue-solver allocation (preflight)
`OPENMX_CUBLAS_MIN_FREE_AFTER_MB`	—	Same as above, for cuBLAS
Opt-in matrix element construction (Set_Hamiltonian.c / Set_OLP_Kin.c)		
`OPENMX_SETHAM_BASE_GPU`	0	Run the base combination of H on the GPU (measurements show the CPU is faster)
`OPENMX_OLPKIN_RADIAL_GPU`	0	Run the radial integrals of overlap and kinetic matrix elements on the GPU (requires thread count 1. Transfer-bound, so the CPU is usually faster)

Table 10-1: GPU-related environment variables

10.3 Tuning Guidelines

1. **Run with the defaults first** — the defaults are conservatively tuned for a configuration where a 16 GB-class GPU is shared by multiple ranks.
2. **Check stderr** — if the "GEMMul8 workspace fallback" warning appears, VRAM shortage has caused a fall back to FP64. Remedies include reducing the number of ranks, lowering `OPENMX_BAND_GPU_TURN_GROUP`, or reducing `NUM_MOD`.
3. **When the GPU can be dedicated** — `OPENMX_BAND_GPU_PERSISTENT=1` and `OPENMX_GPUSOLVER_SERIAL_GPU_TURNS=0` (or a larger TURN_GROUP) may reduce transfers and synchronization.
4. **When reproducibility is the top priority** — pinning GEMM to FP64 with `OPENMX_GEMMUL8_DISABLE=1` eliminates the effect of memory-dependent engine switching (§ 4.8).
5. **DC/DC-LNO** — in systems with small local clusters, few targets qualify for GPU offloading. Lowering `OPENMX_DC_GPU_THRESHOLD` increases the number of targets, but for small matrices the transfer cost tends to dominate.

11. Summary of Constraints and Caveats

11.1 Build and Runtime Environment Assumptions

- **This tree cannot be built as CPU-only** (CUDA headers are included unconditionally). If a CPU version is needed, use the original or `makefile.cpu_intel.bak`.
- NVHPC 26.3 + CUDA 13.1 is the default. When changing versions, re-verify the list of miscompilation workarounds in §6.5.
- GEMMul8 requires INT8 Tensor Cores and is built for a single architecture. Rebuild when changing GPU generations.
- GPU execution is expected to run MPI-only (1 OpenMP thread). The GPU path of `Set_OLP_Kin` explicitly requires `Nthrds==1`.

11.2 Memory Sizing Guidance

- VRAM demand per rank (band calculation, dimension n): several complex dense matrices of $16n^2$ bytes each + syevdx workspace + GEMMul8 workspace (about 1 GiB; complex is roughly 3× the real case).
- On shared GPUs, "number of simultaneous ranks × the above" must fit within free VRAM. The Band paths have automatic control (turn serialization and the adaptive limit), but Cluster root-dense and DC-LNO direct dispatch have no equivalent automatic throttling (DC has a preflight).

11.3 Numerical Reproducibility

- **Factors that can break bitwise reproducibility:** (1) memory-dependent switching between GEMMul8 ⇄ cuBLAS FP64, (2) `acc atomic`/atomicAdd in scatter-add (non-deterministic addition order), (3) path differences in the small-eigenvalue handling of S, (4) differences in GEMMul8 environment variable settings (num_moduli / fastmode).
- **Design choices made for reproducibility:** the sequential `loop seq` reduction in DM construction, GEMMul8's own bitwise reproducibility, the bit-identical Bessel table cache, and (though not wired in) the sequential-reduction kernels of openmx_cuda_kernels.cu.
- For verification, the reliable approach is to fix conditions with `OPENMX_GEMMUL8_DISABLE=1` and ample VRAM headroom.

11.4 Hangs and Abnormal Termination

- `wait_cudafunc` loops forever on a permanent error (§3.6). It is the prime suspect when "GPU execution appears stuck".
- Behavior on eigenvalue computation failure differs by path: abort / CPU fallback / pass-through (Table 5-2).
- The collective device-assignment functions assume all ranks call them together (calling from only one side deadlocks).

11.5 Porting Considerations for AMD and Other Vendors

- The gpusolver naming is in place, but the switch is not mechanized via macros or the like. The cuSOLVER dependency points are concentrated in `gpusolver_Syevd(x).c` and each solver's persistent context (direct

Xsyevdx calls).

- The cuBLAS dependency points are Ddgmm/Zdgmm and the GEMMul8 bridge (internal INT8 GEMM and fallback GEMM). GEMMul8 has an upstream HIP/ROCm backend, but this integration (single-TU instantiation) instantiates only the CUDA path.
- OpenACC depends on NVHPC (for AMD, a rewrite to OpenMP target offload or similar would be needed). The "return value 0 = success" assumption of `wait_cudafunc` is another point to check when porting.

11.6 Non-GPU Behavioral Differences from the Original

- Two heap corruption bugs under OpenMP parallelism have been fixed (§ 8.1) — bugs that also exist in the original.
- The stack soft limit is automatically raised at startup (§ 8.2).
- The runtest pass/fail criteria and execution procedure have been made stricter (§ 8.3).
- The version string is "4.0 GPU".

11.7 Dormant and Preparatory Code (to Avoid Confusion When Modifying)

Code	Status and role
`gpusolver_Syevd` (Xsyevd version)	Zero callers (unified on the Syevdx version). The old-style "global rank % device count" assignment remains
`LU_Zinverse_GPU`	Implementation complete, call site commented out. Groundwork for GPU offloading of the NEGF surface Green's function (§ 5.7)
`openmx_cuda_kernels.cu/.h`	Collection of raw CUDA kernels verified for bitwise reproducibility. Untracked by git, excluded from the build, zero callers (§ 6.4)
`ClusterCol_GpuSolverDensePath`	Unreachable legacy path (the root-dense path gotos first)
GEMMul8 helpers for the DMmu/CWF/LNO families, `BandCol_GpuSolver_Zgemm_OpenACC`, `ClusterCol_GEMMul8Dgemm_OpenACC`	Definitions only (groundwork for future GEMM GPU offloading). Note that `my_cublasDgemm/Zgemm` were removed in `a7f8cba6`
DC-LNO GPU proxy mechanism	Preserved under `DCLNO_ENABLE_SHARED_GPU_PROXY 0` (experimental)
`OutData_Binary.o` / `getDeviceCount()` declaration	Dead link / dead declaration

Table 11-1: Dormant and preparatory code

Appendix A. Changed Files (37 files)

+n/−m are raw diff line counts. For files that underwent full reformatting, the substantive changed lines (whitespace-insensitive) are given in parentheses. Classification: **GPU**=GPU offloading, **OMP**=OpenMP fix, **FIX**=bug fix / hardening, **OPT**=CPU optimization, **BLD**=build, **MISC**=other.

File	+n/−m (substantive)	Class	Change summary
`Band_DFT_Col.c`	+4,116/−2,491 (3,667)	GPU	Full GPU offloading of the collinear band path: dense-matrix construction cache, Xsyevdx diagonalization, GEMMul8 triple products, S-transform cache, GPU-turn serialization, both all_knum paths
`Band_DFT_NonCol.c`	+3,260/−91 (2,736)	GPU	GPU offloading of the non-collinear band path: OpenACC-resident root-dense path, adaptive concurrency limit based on VRAM estimation
`Cluster_DFT_Col.c`	+1,016/−12 (859)	GPU	Root-dense GPU diagonalization path, device cache for the S transform, matrix construction from the packed cache, eigenvector distribution
`Cluster_DFT_NonCol.c`	+1,706/−218 (1,252)	GPU	Root-dense diagonalization (Dsyevx/Zheevx), GEMMul8 × 12, eigenvector cache and scatter, DM/EDM reduction
`Divide_Conquer.c`	+4,399/−2,978 (2,223)	GPU	GPU offloading of the DC method: per-atom cluster Xsyevdx, multi-stage GEMM (GEMMul8→cuBLAS→CPU), VRAM preflight and sticky disabling
`Divide_Conquer_LNO.c`	+1,934/−2,007 (1,879)	GPU	GPU offloading of DC-LNO: direct per-rank dispatch (the proxy mechanism is retained), threshold 800
`Krylov.c`	+366/−16	GPU	Per-thread GPU workspace pool, 4 dgemm call sites → GEMMul8, subspace diagonalization → gpusolver
`Band_DFT_Col_CWF.c` and 9 boilerplate files (§ 7.10)	+63–76/−2–11 each	GPU	Boilerplate GPUSOLVER branch inserted before the ELPA2 branch (Dsyevx/Zheevx via dense_utils)
`Electric_Polarization_BandCol.c` / `_BandNonCol.c`	+27/−5, +40/−14	GPU	GPUSOLVER branch at 5 diagonalization sites in each polarization calculation
`Eigen_lapack.c`	+143/−8	GPU	Branch to gpusolver_Syevdx for n≥1000, added an OpenACC-direct variant. 20+ calling files benefit automatically
`EigenBand_lapack.c`	+112/−3	GPU	Complex version: gpusolver_zheevx branch, Eigen_HH fallback on failure
`Lapack_LU_inverse.c`	+134/−4	GPU	Added LU_Zinverse_GPU using cublasZgetrf/ZgetriBatched (call sites commented out = dormant)
`Set_Hamiltonian.c`	+1,999/−1,149 (1,784)	GPU	OpenACC version of the grid integration (currently disabled) + new packed H/S cache API for GPUSOLVER + memory-based Allreduce decision
`Set_Nonlocal.c`	+1,356/−306	GPU/FIX	OpenACC offloading of the radial k integration + fixed-size arrays moved to the heap (with overflow checks)
`Set_OLP_Kin.c`	+529/−181	GPU/OPT	OpenACC offloading of the radial integration (opt-in) + bit-identical Bessel table cache
`Total_Energy.c`	+319/−115	GPU/OMP	GPU reduction calls for the batched EH0 two-center terms, EXC/EH1, dipole, and CWF-Dc + OpenMP private fix
`Total_Energy_GPU.c` (treated as new)	517 lines	GPU	Collection of OpenACC kernels for Total_Energy (separate TU for a split -acc build)
`Force.c`	+11,522/−11,115 (3,672)	GPU	OpenACC reductions for forces #2/#4/#5 (#4B kept on CPU). Most raw lines come from full re-indentation

File	+n/−m (substantive)	Class	Change summary
`Stress.c`	+1/−1	OMP	Added i to the OpenMP private list (heap corruption fix)
`DFT.c`	+172/−4	GPU	GPU device initialization, OpenACC executing-rank selection, packed-cache preparation, NonCol eigenvector scatter
`Input_std.c`	+15/−3	GPU	Added gpusolver to scf.eigen.lib (cusolver is a deprecated alias), added scf.Gpu.Num
`openmx_common.h`	+93/−13	GPU/MISC	CUDA header inclusion, GPUSOLVER enum, GPU_CPU_SWITCH_NUM, wait_cudafunc/cudacall, GPU-related prototypes, include guard, Version "4.0 GPU"
`tran_prototypes.h`	+10/−0	GPU	Added the CUDA_CHECK macro
`openmx.c`	+16/−0	FIX	Raise_Stack_Limit() (automatic stack-limit raising)
`Runtest.c`	+19/−7	FIX	Stricter .dat detection, deletion of stale .out files before tests
`Free_Arrays.c`	+23/−17	GPU/FIX	GEMMul8 workspace release at shutdown + NULL assignment after free
`truncation.c`	+14/−10	FIX	NULL assignment after freeing the VNA/VEF grids (double-free prevention)
`Spherical_Bessel.c`	+12/−24	OPT	Removed per-call malloc/free from the hot path
`mpao.c`	+0/−1	MISC	One-line include difference only (not part of the build)
`makefile` → `Makefile`	+360/−374	BLD	Complete rewrite from an Intel oneAPI + MKL configuration to NVHPC + CUDA + bundled FFTW + GEMMul8 (Chapter 9)

Note: `Set_ProExpn_VNA.c` was GPU-offloaded once during development and later reverted to the CPU version (commits 17→31); in the current tree it has no difference from the original. Everything under `elpa-2018.05.001/` is also fully identical. No source files were deleted.

Appendix B. Newly Added Files

B.1 New Sources (13 files, 2,555 lines total)

File	Lines	Role
`gpusolver_Syevdx.c`	387	Partial eigendecomposition wrapper based on cusolverDnXsyevdx (4 variants: real/complex × host/OpenACC). The core of GPU diagonalization
`gpusolver_Syevd.c`	131	cusolverDnXsyevd wrapper variant (derived from the NVIDIA CUDA Library Samples). Currently unused
`openmx_gpusolver_dense_utils.h`	79	Common dispatch helper for dense diagonalization (0/1-based conversion + MPI_Abort on failure). Used by 11 files
`gemmul8_bridge.cu`	393	GEMMul8 call bridge: workspace cache, environment-variable control, cuBLAS fallback
`gemmul8_openmx.cu`	67	Single-TU explicit instantiation of the GEMMul8 templates (only 4 symbols)
`Total_Energy_GPU.c`	517	Collection of OpenACC kernels for the total-energy calculation (EXC/EH1, dipole, CWF-Dc, batched EH0)
`set_cuda_default_device_from_local_rank.c/.h`	73+9	Round-robin CUDA device assignment by intra-node local rank
`set_openacc_device_from_local_rank.c/.h`	79+12	OpenACC version of the same (acc_set_device_num)
`LU_Zinverse_GPU.h`	9	Declaration of the GPU complex LU inverse (dormant)
`openmx_cuda_kernels.cu/.h`	696+103	Verified kernel collection converting OpenACC regions to raw CUDA (not wired in, excluded from the build; § 6.4)

Note: the generic GEMM wrappers `my_cublasDgemm.c` / `my_cublasZgemm.c` that once existed (49+63 lines, no callers) were deleted in commit `a7f8cba6` (§ 3.5).

B.2 Build and Documentation Files

File	Role
`Makefile` (1,373 lines)	The new build system itself. Includes commented configuration examples for various supercomputers. Defaults to NVHPC 26.3 + CUDA 13.1
`Makefile.bunshiken` (1,366 lines)	Variant for a shared compute system (NVHPC 25.9-nompi + OpenMPI 5.0.8 + gcc-toolset-15). One generation older configuration
`makefile.cpu_intel.bak` (1,383 lines)	Byte-identical backup of the original makefile (Intel CPU configuration)
`third_party/`	GEMMul8 (submodule, pinned to commit 884a287), fftw3 (submodule, tag fftw-3.3.11), fftw-3.3.11.tar.gz (to supplement codelets)

Appendix C. Development History (all 51 commits)

#	Hash	Title (as committed)
1	`0cf2acbc`	Initial commit
2	`60b6aa1e`	Performed GPU acceleration
3	`2f94ca06`	Performed GPU acceleration on Divide_Conquer_LNO.c
4	`e3fc656e`	Optimized Band_DFT_NonCol.c
5	`24e8b34f`	Optimized Cluster_DFT_Col.c
6	`33e9459b`	Optimized Cluster_DFT_Col.c
7	`82579d12`	Bug fix
8	`33a07450`	Fixed bugs in Band_DFT_Col.c and Cluster_DFT_Col.c
9	`b4221bba`	Fixed a bug in runtestL
10	`18d92700`	Bug fix
11	`563f69bd`	Add files that were not included in the commit
12	`1b020264`	Optimized Band_DFT_Col.c and Band_DFT_NonCol.c
13	`f4041307`	Optimized Set_Hamiltonian.c
14	`9f1633da`	Optimized Set_OLP_Kin.c
15	`d6b463a8`	Explicitly indicate that the generation of overlap matrices is GPU-accelerated
16	`5383544b`	Optimized Total_Energy.c
17	`c9e111ea`	Optimized Set_ProExpn_VNA.c
18	`8162b8ec`	Optimized Set_ProExpn_VNA.c and Total_Energy.c
19	`91416fb1`	Bug fix
20	`24a113a9`	Add files that were not included in the commit
21	`87ea7062`	Bug fix
22	`faf76fa2`	Bug fix
23	`a8224724`	Add .gitignore for build artifacts and runtime output
24	`d3c45cb3`	Optimized Bnad_DFT_NonCol.c
25	`db7b9327`	Add files that were not included in the commit
26	`a61f24b3`	Bug fix
27	`d6d19f99`	Add README.md
28	`a91f920f`	Bug fix
29	`88b135f4`	Optimized Krylov.c
30	`34cf8bc1`	Optimized Divide_Conquer_LNO.c
31	`685a2bc9`	Reverted Set_ProExpn_VNA.c to the CPU version
32	`2aa31a43`	Add files that were not included in the commit
33	`d3effce1`	Bug fix of Divide_Conquer.c
34	`34b52269`	GPU acceleration for paths where all_knum !=1 in Band_DFT_Col.c
35	`75ef67c7`	GPU acceleration for paths where all_knum !=1 in Band_DFT_NonCol.c
36	`1e0c777e`	Remove runtest.result from Git

#	Hash	Title (as committed)
37	`c8c1e3e1`	Modify the Makefile
38	`cb4d348d`	Now compatible with the latest GEMMul8
39	`d451eeb8`	Fixed OpenMP bugs in Total_Energy.c and Stress.c
40	`2302f965`	The name has been changed from CuSOLVER to GPUSOLVER
41	`d5b897fd`	Band_DFT_Col: serialize GPU turns in pairs and release GEMMul8 workspace per turn
42	`788acb4c`	Delete unnecessary files, such as the MAGMA library
43	`d8219384`	Rename the identifier from cusolver to gpusolver
44	`8647a5ed`	Set large2_example and Large3_example to be processed on the GPU.
45	`f508a2a6`	Enable direct GPUSOLVER dispatch for DC-LNO
46	`3545a9d5`	Edit README.md
47	`ca3d97c2`	Edit README.md
48	`43e30c65`	Limit band GPU solver concurrency adaptively
49	`66ce1e62`	Use GEMMul8 for remaining GPU GEMM paths
50	`a7f8cba6`	Remove obsolete cuBLAS wrapper files
51	`8c7801fa`	Limit band GPU k-turn concurrency

References

1. OpenMX official site: `http://www.openmx-square.org/` (code, manual, input keywords)
2. GEMMul8 (GEMMulate) repository: `https://github.com/RIKEN-RCCS/GEMMul8` (MIT license, lead developer Yuki Uchino, RIKEN R-CCS)
3. Y. Uchino, K. Ozaki, T. Imamura, "High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines", SC Workshops '25, doi:10.1145/3731599.3767539
4. K. Ozaki, Y. Uchino, T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", arXiv:2504.08009
5. Y. Uchino, S. Ma, T. Imamura, K. Ozaki, T. Gutsche, "Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem", arXiv:2512.08321 (theory behind the complex support)
6. Y. Uchino, K. Ozaki, T. Imamura, "Double-Precision Matrix Multiplication Emulation via Ozaki-II Scheme with FP8 Quantization", arXiv:2603.10634
7. H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", Int. J. High Perform. Comput. Appl. (2024), doi:10.1177/10943420241239588 (prior work in the ozIMMU / Ozaki Scheme I line)
8. NVIDIA cuSOLVER Documentation (cusolverDnXsyevd/Xsyevdx): `https://docs.nvidia.com/cuda/cusolver/`
9. NVIDIA cuBLAS Documentation: `https://docs.nvidia.com/cuda/cublas/`
10. NVIDIA HPC SDK Documentation (OpenACC): `https://docs.nvidia.com/hpc-sdk/`
11. ELPA (Eigenvalue soLvers for Petaflop Applications): `https://elpa.mpcdf.mpg.de/`
12. FFTW: `https://www.fftw.org/`

About this document: This document was prepared based on an examination of the source tree as of 2026-07-05 (commit `8c7801fa`), a full-file diff against the original OpenMX 4.0, and the complete commit history of the development repository. Line numbers and default values cited herein refer to that revision. If any discrepancy is found, the source code should take precedence as the primary reference.